



Tribhuvan University

Faculty of Humanities and Social Sciences

PROJECT REPORT ON

MediVision (Hospital Management System)

Submitted to

Department of Computer Application

Nepal Kasthamandap College

Kalanki, Kathmandu

In partial fulfillment of the requirements for the Bachelors in Computer Application

Submitted by

Rubi K.C

TU Registration: 6-2-144-25-2020

November 2025

Under the Supervision of

Shyam Maharjan



Tribhuvan University

Faculty of Humanities and Social Sciences

Nepal Kasthamandap College

Kalanki, Kathmandu

Bachelor in Computer Applications (BCA)

Supervisor's Recommendation

I hereby recommend that this project prepared under my supervision by **Rubi K.C** entitled **“MediVision (Hospital Management System)”** in the Partial Fulfillment of requirement for the degree of Bachelor in Computer Application is recommended for that final evaluation.

Shyam Maharjan

Project Supervisor

BCA Department

Nepal Kasthamandap College



Tribhuvan University

Faculty of Humanities and Social Sciences

Nepal Kasthamandap College

Kalanki, Kathmandu

Bachelor in Computer Applications (BCA)

Letter of Approval

This certify that this project prepared by **Rubi K.C** entitled “**MediVision (Hospital Management System)**” in the Partial Fulfillment of requirement for the degree of Bachelor in Computer Application has been evaluated. In our opinion it is satisfactory in the scope and quality as a project for the required degree.

Name: Shyam Maharjan Supervisor BCA Department Nepal Kasthamandap College Kalanki, Kathmandu	Name: Er. Sujan Poudel Head of Department BCA Department Nepal Kasthamandap College Kalanki, Kathmandu
Name: Roshan Poudel Internal Examiner BCA Department	Name: Roshan Tandukar External Examiner Patan Multiple Campus

Abstract

MediVision is a smart hospital management system developed using the MERN stack (MongoDB, Express.js, React.js, and Node.js) with the goal of making healthcare services more efficient, accessible, and user-friendly. The system allows patients to enter their symptoms and receive an early prediction of possible diseases using the Logistic Regression algorithm. This feature helps users gain quick awareness about potential health problems and encourages timely medical consultation.

A doctor recommendation feature is powered by the Bayesian Ranking Algorithm, which suggests the most suitable doctors based on their specialization, ratings, and patient reviews. This intelligent ranking system builds trust and supports better decision-making for patients. An integrated appointment booking function further simplifies the process, reducing long waiting times and eliminating unnecessary manual work.

MediVision also provides strong management support for doctors and administrators. Doctors can view and manage their appointments, update their profiles, and upload medical reports for patients, while patients can securely access and download these reports whenever needed. Administrators handle user accounts, doctor data, and appointment management through a centralized dashboard. Overall, the system combines smart prediction, reliable recommendations, and efficient data handling to offer a modern and effective digital healthcare solution.

Keywords: Hospital Management System, Disease Prediction, Logistic Regression, Bayesian Ranking Algorithm, Doctor Recommendation.

Acknowledgement

I would like to express my special thanks of gratitude to our supervisor Shyam Maharjan who gave me the wonderful opportunity to work on this project titled MediVision (Hospital Management System). Through this project, I learned about various new tools, technologies, and practical applications of machine learning in the healthcare field, for which I am truly thankful.

I would also like to express my heartfelt gratitude to our BCA Program Coordinator Er. Sujan Poudel for his continuous support, encouragement, and guidance throughout the completion of this project.

I would like to thank my parents and friends for their constant motivation, help, and understanding, which made it possible to complete this project within the given time frame. Finally, I am grateful to all those who have directly or indirectly contributed to the successful completion of my project.

Yours sincerely,

Rubi K.C

Table of Contents

Supervisor’s Recommendation	i
Letter of Approval.....	ii
Abstract.....	iii
Acknowledgement	iv
List of Figures.....	vii
List of Tables.....	viii
List of Listings	ix
List of Abbreviations.....	x
Chapter 1: Introduction	1
1.1 Introduction.....	1
1.3 Objectives.....	2
1.4 Scope and Limitation	2
1.4.1 Scope.....	2
1.4.2 Limitations.....	2
1.5 Development Methodology	3
1.6 Report Organization.....	3
Chapter 2: Background Study and Literature Review	5
2.1 Background Study.....	5
2.2 Literature review	5
Chapter 3: System Analysis and Design.....	9
3.1 System Analysis	9
3.1.1 Requirement Analysis.....	9
3.1.2 Object Modeling using class and object diagram.....	13
3.1.3 Dynamic Modeling using sequence diagram	16
3.2.3 Process Modeling using Activity diagram.....	17

3.2 System Design.....	19
3.2.1 Refinement of Class diagram	19
3.2.2 Component Diagram.....	21
3.2.3 Deployment Diagram	24
3.2.4 Wireframing.....	24
3.2.5 Interface Design (UI Interface/Interface Structure Diagrams)	27
3.3 Algorithm Details	28
3.3.1 Logistic Regression Algorithm	28
3.3.2 Bayesian Ranking Algorithm.....	30
Chapter 4: Implementation and Testing.....	33
4.1 Implementation.....	33
4.1.1 Tools Used	33
4.1.2 Implementation Details of Modules.....	34
4.2 Testing	38
4.2.1 Test Cases for Unit Testing.....	38
4.2.2 Test Cases for System Testing	39
4.3 Result Analysis.....	43
Chapter 5: Conclusion and Future Recommendations	47
5.1 Conclusion.....	47
5.2 Outcome	47
5.3 Future Recommendations.....	48

References

Appendices

List of Figures

Figure 1.1: Agile Methodology	3
Figure 3.1: Use case Diagram of MediVision.....	10
Figure 3.2: Class diagram for MediVision.....	14
Figure 3.3: Object diagram for MediVision.....	15
Figure 3.4: Sequence diagram for MediVision	16
Figure 3.5: Activity diagram for Admin for MediVision.....	17
Figure 3.6: Activity diagram for Patient for MediVision.....	18
Figure 3.7: Activity diagram for Doctor for MediVision.....	19
Figure 3.8: Refinement class diagram for MediVision.....	20
Figure 3.9: Component diagram for Doctor of MediVision	21
Figure 3.10: Component diagram for Admin of MediVision	22
Figure 3.11: Component diagram for Patient of MediVision	23
Figure 3.12: Deployment diagram of MediVision.....	24
Figure 3.13: Wireframing Design of Homepage.....	25
Figure 3.14: Wireframing Design of User Registration Page	25
Figure 3.15: Wireframing Design of Doctor Dashboard	26
Figure 3.16: Wireframing Design of Admin Dashboard.....	26
Figure 3.17: Interface Design of Homepage.....	27
Figure 3.18: Interface Design of Health Predict Page	27
Figure 3.19: Interface Design of Appointment Booking Page.....	28
Figure 4.1: Top-Rated Doctor Using Bayesian Ranking	44
Figure 4.2: Recommended Doctors for Patient of MediVision	45
Figure 4.3: Health Prediction Using Logistic Regression Algorithm	46

List of Tables

Table 3.1: Technical Feasibility Of Medivision.....	11
Table 3.2: Project Schedule.....	12
Table 4.1: Unit Testing for MediVision	38
Table 4.2: System Testing for MediVision.....	39
Table 4.3: Data Collection Test for MediVision	40
Table 4.4: Data Preprocessing Test for MediVision.....	41
Table 4.5: Model Architecture for MediVision.....	42
Table 4.6: Model Training Test Results for MediVision.....	42
Table 4.7: Disease Prediction Evaluation for MediVision.....	43

List of Listings

Listing 4.1: Code Snippet for Patient Registration	34
Listing 4.2: Code Snippet for Patient Login	34
Listing 4.3: Code Snippet for Bayesian Ranking Algorithm.....	35
Listing 4.4: Code Snippet for Logistic Regression Algorithm	35
Listing 4.5: Code Snippet to Book an Appointment	35
Listing 4.6: Code Snippet for Patient Report Management	36

List of Abbreviations

Abbreviations	Description
AI	Artificial Intelligence
CSS	Cascading Style Sheet
DB	Database
ML	Machine Learning
MERN	MongoDB, Express.js, React, and Node.js
MV	MediVision
OOP	Object Oriented Programming
UI	User Interface

Chapter 1: Introduction

1.1 Introduction

The healthcare sector is rapidly advancing with the help of digital technologies to improve patient care and reduce the burden of manual administrative tasks. In a hospital setting, managing processes like patient registration and appointment scheduling can become time-consuming and prone to errors when done manually. To address these challenges, MediVision offers a smart and integrated Hospital Management System that brings together various hospital operations on a single, unified platform, helping improve both efficiency and patient experience.

MediVision is a powerful web app built with the MERN stack (MongoDB, Express.js, React.js, and Node.js) that offers a seamless and intuitive experience for users. One of its standout features is the disease prediction module, which uses machine learning algorithms to analyze patient-reported symptoms and predict possible health conditions. This allows patients to gain early insights into their health status and make informed decisions when booking doctor appointments, thereby supporting faster and more accurate care.

The system includes three levels of user access: patients, doctors, and administrators. Patients can log in, enter their symptoms, receive disease predictions, and book appointments accordingly. Doctors can manage their schedules, track earnings, and update their profiles. Admins oversee the entire system, managing doctor accounts and appointments to ensure smooth operation. With its intelligent features and automated workflow, MediVision is designed to support smarter, more proactive healthcare delivery.

1.2 Problem Statement

In Nepal's healthcare system, numerous challenges hinder patients' ability to receive timely, informed, and efficient medical attention. These challenges include a lack of early disease awareness, limited access to intelligent health guidance, inefficient appointment systems, manual handling of administrative tasks, and poor coordination between patients, doctors, and hospital staff. The absence of predictive tools and personalized support further complicates the patient journey, delaying diagnosis and treatment.

The proposed MediVision project addresses the following key problems:

- Patients often struggle to understand their symptoms or predict potential illnesses in the early stages.

- There is no centralized platform offering intelligent and personalized healthcare support tailored to individual symptoms and needs.
- The process of booking appointments and managing healthcare information is still inefficient and largely manual.

MediVision is developed to tackle these pressing issues by offering a smart, user-friendly and centralized platform that connects patients and doctors. By integrating symptom-based disease prediction with hospital management tools, MediVision aims to modernize and streamline the healthcare experience in Nepal empowering users with early insights, efficient scheduling, and better access to care.

1.3 Objectives

Objectives of the project which are mainly focused are mentioned below:

- To develop a web-based platform where patients can easily input symptoms and receive disease predictions.
- To provide an online appointment system that allows patients to book consultations with doctors based on predicted health conditions.

1.4 Scope and Limitation

MediVision is a web-based application that offers disease prediction based on patient symptoms and simplifies doctor appointment booking for a smoother healthcare experience.

1.4.1 Scope

The scope of MediVision is to improve access to healthcare and streamline hospital services through a smart online platform:

- It allows patients to enter symptoms and get disease predictions.
- It helps patients book appointments with suitable doctors based on their predicted condition.
- It allows doctors to manage appointments and update their profiles.
- It allows doctors to upload medical reports, which patients can view and download.

1.4.2 Limitations

While MediVision aims to offer a smart healthcare solution, it has certain limitations:

- It doesn't provide real-time chat or video consultation with doctors.
- It does not send SMS reminders or push notifications for appointments or follow-ups.

1.5 Development Methodology

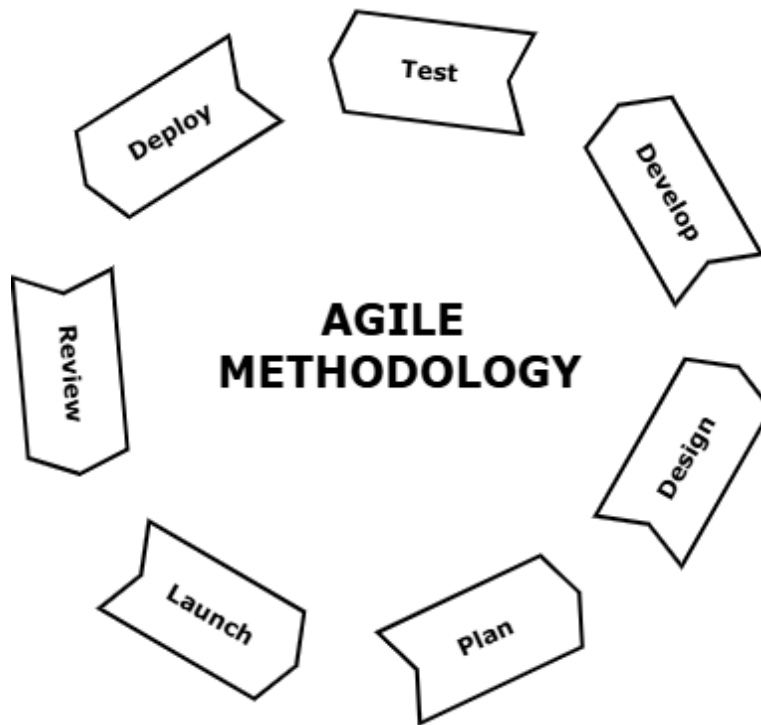


Figure 1.1: Agile Methodology

Agile methodology is used for MediVision because it allows the project to be developed in small, manageable steps, making it easier to add new features and refine them based on feedback. This approach encourages continuous testing, quick adjustments, and teamwork throughout the process. By focusing on user needs and gradual improvements, Agile helps ensure that MediVision remains flexible, reliable, and user-friendly.

1.6 Report Organization

Introduction

Medivision connects patients, doctors, and admin in a single platform, clearly defining the objectives and requirements of the system.

Background Study and Literature Review

Existing healthcare systems were studied, their limitations analyzed, and Medivision was designed to improve them with better features and functionality.

System Analysis and Design

This section defines the system's functional and non-functional requirements, use cases, and overall design, including data models, database schema, and architecture.

Implementation and Testing

Medivision was developed using modern tools and technologies, and the system was tested with different cases to ensure smooth and reliable performance.

Conclusion and Future Recommendation

The project is summarized with its outcomes, while future improvements such as AI-based diagnosis, video consultations, and wearable device integration are recommended.

Chapter 2: Background Study and Literature Review

2.1 Background Study

Today, healthcare all around the world is slowly moving toward digital systems. With better internet access and the growth of artificial intelligence, many hospitals and health organizations are now using online platforms and smart tools to make their services faster and more reliable. These digital solutions help reduce paperwork, support better decision-making, and improve communication between patients and doctors. As a result, people can now check symptoms, book appointments, and connect with healthcare providers more easily than before. [1]

In Nepal, the healthcare system includes both public and private services, but it still faces many difficulties. In rural and remote areas especially, people often have trouble getting timely treatment or even understanding their health problems at an early stage. Many hospitals still depend on manual record-keeping and traditional methods, which can cause delays, confusion, and long waiting times for patients. [2]

Because of these problems, digital health platforms can play a very important role in Nepal's healthcare system. An online system can help people receive basic health guidance, find suitable doctors, and schedule appointments without needing to visit the hospital multiple times. At the same time, it can make hospital work more organized and efficient by reducing manual tasks and improving data management. [3]

Keeping these needs in mind, the MediVision system is being developed to support a more modern and user-friendly healthcare experience in Nepal. It brings together important services like symptom analysis, disease prediction, appointment booking, and doctor management within a single web-based platform. By providing tools for both patients and healthcare workers, MediVision aims to make the overall healthcare process smoother, faster, and more convenient for everyone. [4]

2.2 Literature review

This is a web-based hospital management system aimed at improving the way patients, doctors, and administrators interact within a healthcare setting. Developed using technologies such as PHP, MySQL, and JavaScript, the system offers a user-friendly online platform where patients can easily register and book appointments with their preferred doctors. Doctors can efficiently manage their schedules, review patient appointments, and

provide prescriptions through the system. Meanwhile, administrators have the ability to oversee and update doctor profiles, monitor appointment bookings, and manage patient information securely. By digitizing these key hospital processes, the system significantly reduces the manual workload, minimizes errors, and enhances overall operational efficiency. This streamlined approach makes hospital management more organized and convenient for all users, contributing to better patient care and smoother administrative workflows. [5]

The challenges of managing large and complex patient medical records at Sule Lamido University Clinic by developing a web-based application. The system enables both clinicians and patients to access electronic health records easily, reducing the inefficiencies and costs associated with manual record-keeping. It helps solve issues such as inconsistent records, delayed storage and retrieval, and lack of integration between clinic units. By providing complete, reliable, and accessible information, the system supports better decision-making and improves overall clinic operations. The development process followed the Software Development Life Cycle (SDLC), and after testing and validation, the system was found to meet all specified requirements and objectives successfully. [6]

This research explores the use of data mining and machine learning techniques in the healthcare sector to improve early disease prediction and patient management. The study analyzed a large dataset of 4,920 patient records, which included diagnoses of 41 different diseases. From these records, 95 important symptoms were selected and optimized from an initial set of 132 variables to better understand their relationship with specific diseases. The disease prediction system was developed using popular machine learning algorithms such as Decision Tree, Random Forest, and Naïve Bayes classifiers. The paper presents a comparative analysis of the accuracy and effectiveness of these algorithms in predicting diseases. By leveraging these techniques, the system aims to support physicians in diagnosing illnesses early, improving patient outcomes and enhancing healthcare services. [7]

As healthcare becomes more digital, having a safe and efficient way to manage doctor appointments is more important than ever. This study introduces a secure online doctor appointment booking system designed to make it easier for patients to schedule visits while keeping their personal and medical information safe. The system allows patients to book appointments through a simple, user-friendly interface, while also using strong security features like login authentication, encrypted data, and role-based access for doctors and patients. It also includes real-time appointment tracking and notifications to avoid double

bookings and improve the overall experience. By solving common problems like data privacy concerns, identity theft, and appointment confusion, this system aims to support both patients and healthcare providers with a reliable and modern solution. The platform was tested thoroughly to make sure it works well, is secure, and can handle growth making it a practical fit for today's healthcare needs. [8]

Cardiovascular diseases (CVD) have become the leading cause of death globally, highlighting the urgent need for early detection and continuous care. However, constant medical supervision isn't always possible due to time, resource, and expertise constraints. To help address this, machine learning (ML) has emerged as a powerful tool for predicting heart disease by analyzing patient data and identifying risks in advance. This study explores the development of a heart disease prediction system using ML techniques such as the Decision Tree algorithm and the K-means Elbow method for user registration and notifications. MySQL is used for managing reminders like immunization alerts. By leveraging these technologies, the system aims to support doctors in making more accurate and timely diagnoses, reduce manual workload, and ultimately improve patient outcomes through early intervention. [9]

In today's healthcare world, technology like wearable devices and smart medical equipment has made it possible to monitor patients' health in real time making the system smarter and more responsive. However, with these advancements come serious security risks. Hackers could tamper with devices, send false health data, or even disrupt treatment during emergencies. To tackle this issue, a system called HealthGuard was developed. It uses machine learning to detect suspicious or harmful activities in smart healthcare environments. HealthGuard keeps an eye on vital signs collected from different devices and analyzes them to figure out whether something unusual is happening like a real health emergency or a potential cyberattack. It uses machine learning methods like Artificial Neural Networks, Decision Trees, Random Forests, and k-Nearest Neighbor to do this effectively. After being tested on data from eight medical devices and various health and activity scenarios, HealthGuard showed impressive results, detecting threats with 91% accuracy. This system proves how important machine learning can be in not only improving care but also protecting patients in modern healthcare settings. [10]

As online appointment and hospital service systems become more widely used, studies have shown that they greatly improve patient access and satisfaction. Many users report being happier with these systems because they make it easier to schedule appointments, reduce waiting times, and allow bookings to be made from anywhere. These findings show that

having a simple, clear, and easy-to-use design is very important. For a system like MediVision, this means creating an interface that patients can easily understand and use without confusion. [11]

The introduction of web-based Electronic Health Record (EHR) systems in hospitals has highlighted how important planning and staff training are for successful implementation. By switching from paper records to digital ones, hospitals have been able to reduce paperwork, access information more quickly, and improve communication between healthcare workers. However, some challenges, such as resistance from staff, technical limitations, and difficulties in transferring old data, were also faced. This shows that technology alone is not enough, hospitals must also be prepared and willing to adapt in order for systems like MediVision to work effectively in the long term. [12]

Another study that focused on designing a web-based EHR system showed that involving users in the design process makes the system more accurate and efficient. The system included different sections for patients, staff, and administrators, with secure login features and controlled access. After using the system, healthcare workers noticed an improvement in their daily tasks and spent less time on documentation. This supports the idea that MediVision should use a modular design, strong security measures, and regular user testing to ensure smooth performance. [13]

Recent studies on hospital management systems show that bringing different hospital services onto one platform can greatly improve efficiency. By connecting departments like registration, laboratory, pharmacy, and billing, hospitals can reduce errors, prevent data duplication, and deliver services faster. These studies also stress the importance of strong data security, such as encrypted databases and controlled access, to keep patient information safe. This aligns closely with MediVision's goal of providing secure, efficient, and well-organized healthcare services. [14]

Chapter 3: System Analysis and Design

3.1 System Analysis

The system analysis of the system is done by conducting requirement analysis and feasibility analysis as follows:

3.1.1 Requirement Analysis

a. Requirement Analysis

The requirement analysis of MediVision is done through finding the functional and non-functional requirements for the system.

Functional Requirements

For Patient:

- The system should allow patients to sign up, log in, and log out securely.
- The system should allow patients to enter and update personal.
- The system should allow patients to view doctor profiles, availability, and specialties.
- The system should allow patients to book or cancel appointments with doctors.
- The system should allow patients to input symptoms and receive disease predictions.
- The system should allow patients to view the list of healthcare services provided on the website.
- The system should allow patients to view and download medical reports uploaded by doctors.

For Doctor:

- The system should allow doctors to log in and log out securely.
- The system should allow doctors to view and manage patient appointments.
- The system should allow doctors to manage their availability.
- The system should allow doctors to upload medical reports for patients.
- The system should allow doctors to update their profiles.

For Admin:

- The system should allow the admin to log in and log out securely.
- The system should allow the admin to add, update, or remove doctor profiles.
- The system should allow the admin to monitor appointments and system activity logs.
- The system should allow admin to manage doctor's specializations and departments.

- The system should allow admin to manage doctor's availability.
- The system should allow the admin to add, update, and remove healthcare services displayed on the website.
- The system should allow the admin to view all uploaded medical reports (for monitoring purposes).

Use Case Diagram

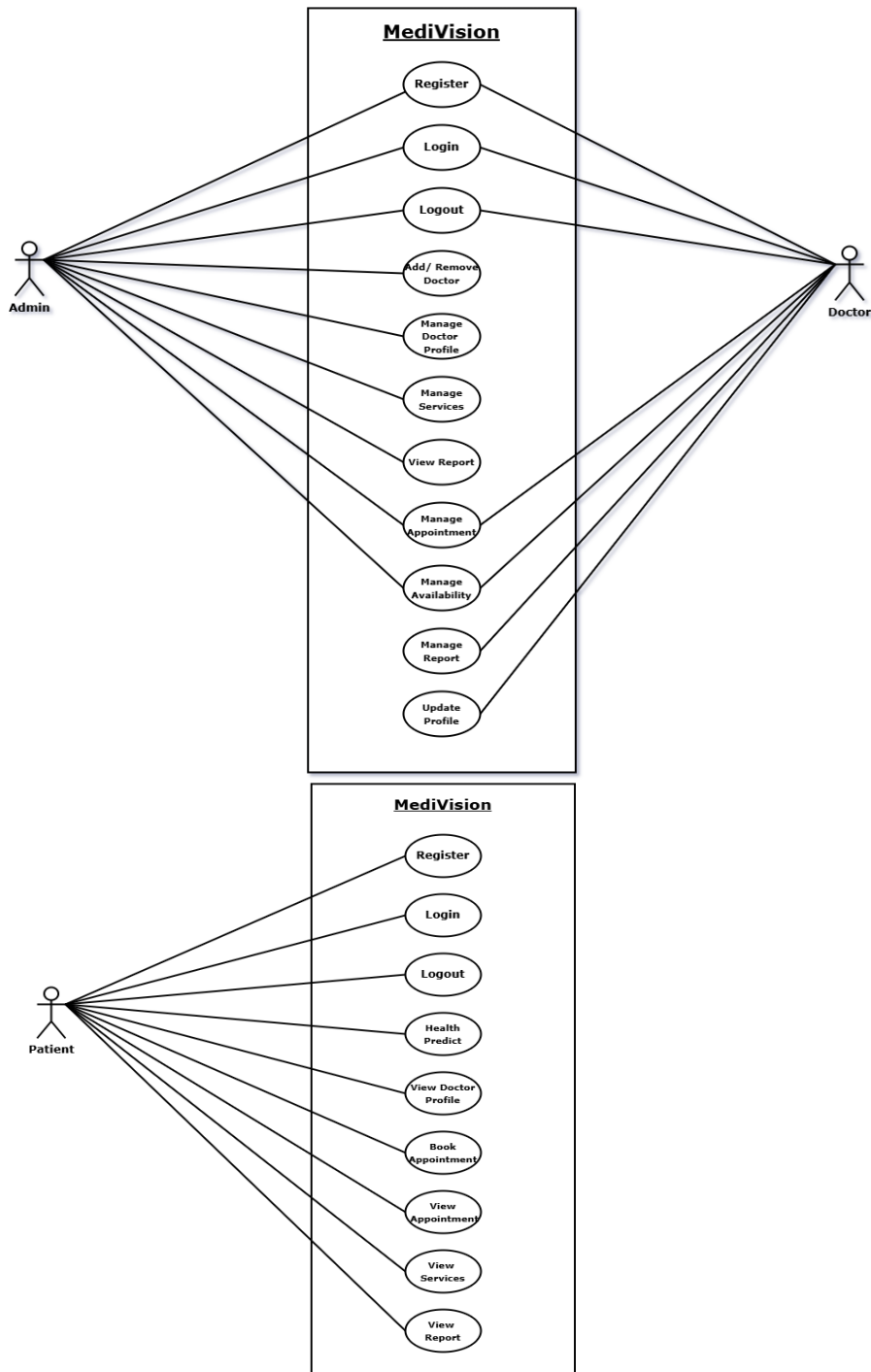


Figure 3.1: Use Case Diagram of MediVision

MediVision is designed to make healthcare easier for everyone. Patients can simply sign up, check doctors, enter their symptoms, book appointments, and later view or download their medical reports. Doctors can manage their schedules, see their patients' appointments, and upload reports after check-ups. The admin takes care of the system by managing doctors, services, departments, and keeping track of reports and appointments, making sure everything runs smoothly.

A. Non-Functional Requirements

Security:

The users need to authenticate themselves before accessing by simply logging in using the email and password that they have used while registering their account. The passwords are also md5 encrypted.

Availability:

Users can access the system at any time they want and the system also runs in multiple browsers such as: Microsoft edge, Chrome, etc.

B. Feasibility Study

Technical Feasibility

The technical feasibility use in this project is mention below:

Table 3.1: Technical Feasibility of MediVision

Technical Knowledge	Hardware Requirements	Software Requirements	Algorithm Used
MongoDB	Laptop	Visual Studio	Logistic Regression
Express.js		Draw.io	Bayesian Ranking
React.js			
Node.js			
Python (Flask)			

The basic programming language which is suitable for project is available and the libraries required for project is capable of achieving the result that we are aiming for. This software which we are going to be used while making this project are freely available and technical skills are manageable so it is technical Feasible.

Operational Feasibility

MediVision is easy to use for anyone with basic internet knowledge. Patients can input symptoms, get disease predictions, and book appointments. Doctors can manage schedules, give prescriptions, and track earnings. Admins can oversee users and system activities, making the platform practical and efficient for daily healthcare use.

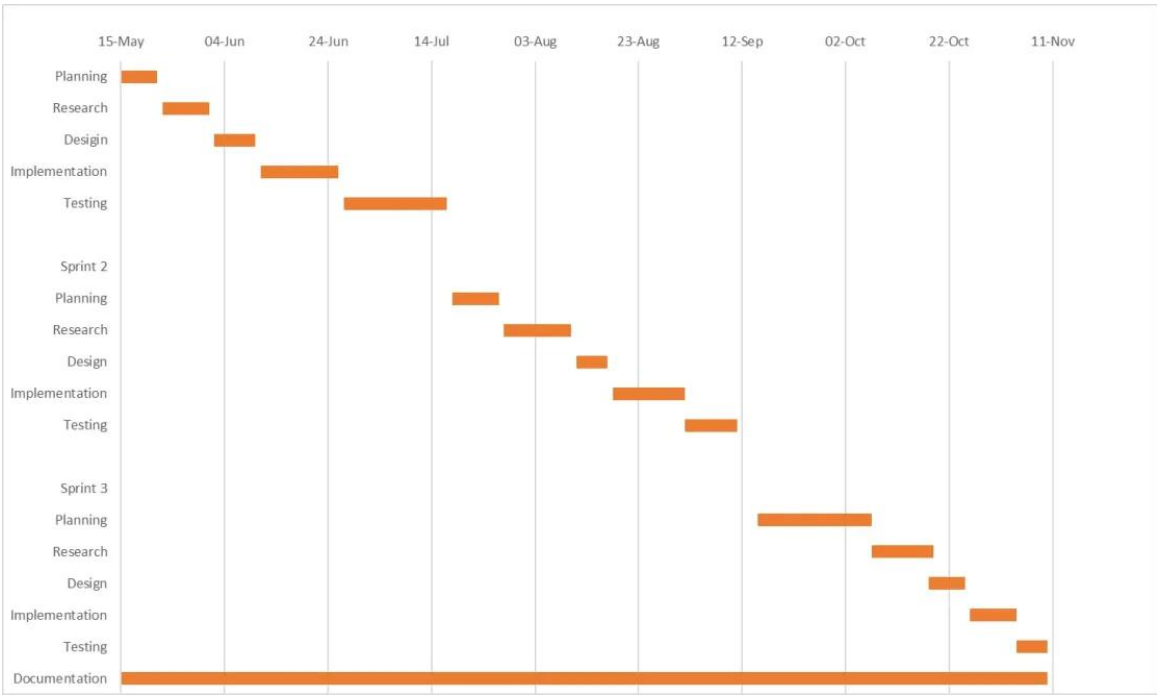
Economic Feasibility

This is academic project so there is no need of financial funding thus there is no need of economic feasibility.

Schedule Feasibility

The system is completed within scheduled time and don't exceed the scheduled time. It visually depicts the project's phases and milestones, enabling effective project management and tracking progress:

Table 3.2: Project Schedule



A Gantt chart is a visual project planning tool that outlines the timeline of activities involved in the development of MediVision. It helps manage and monitor tasks such as patient data collection, symptom analysis integration, doctor profile setup, system design, backend development using MERN stack, and appointment/payment module implementation. By displaying the start and end dates of each activity, the Gantt chart ensures clear visibility of project milestones, highlights overlapping tasks, and helps in identifying potential delays. This enables the team to efficiently track progress and maintain smooth coordination across all phases of the project lifecycle.

3.1.2 Object Modeling using class and object diagram

As the project was decided to be done through the OOP paradigm, the Class diagram of the project was drawn which overall represented the major classes, their attributes and methods, and relationships among the classes. The class diagram drawn helped us by providing a blueprint for the coding implementation in the coding phase.

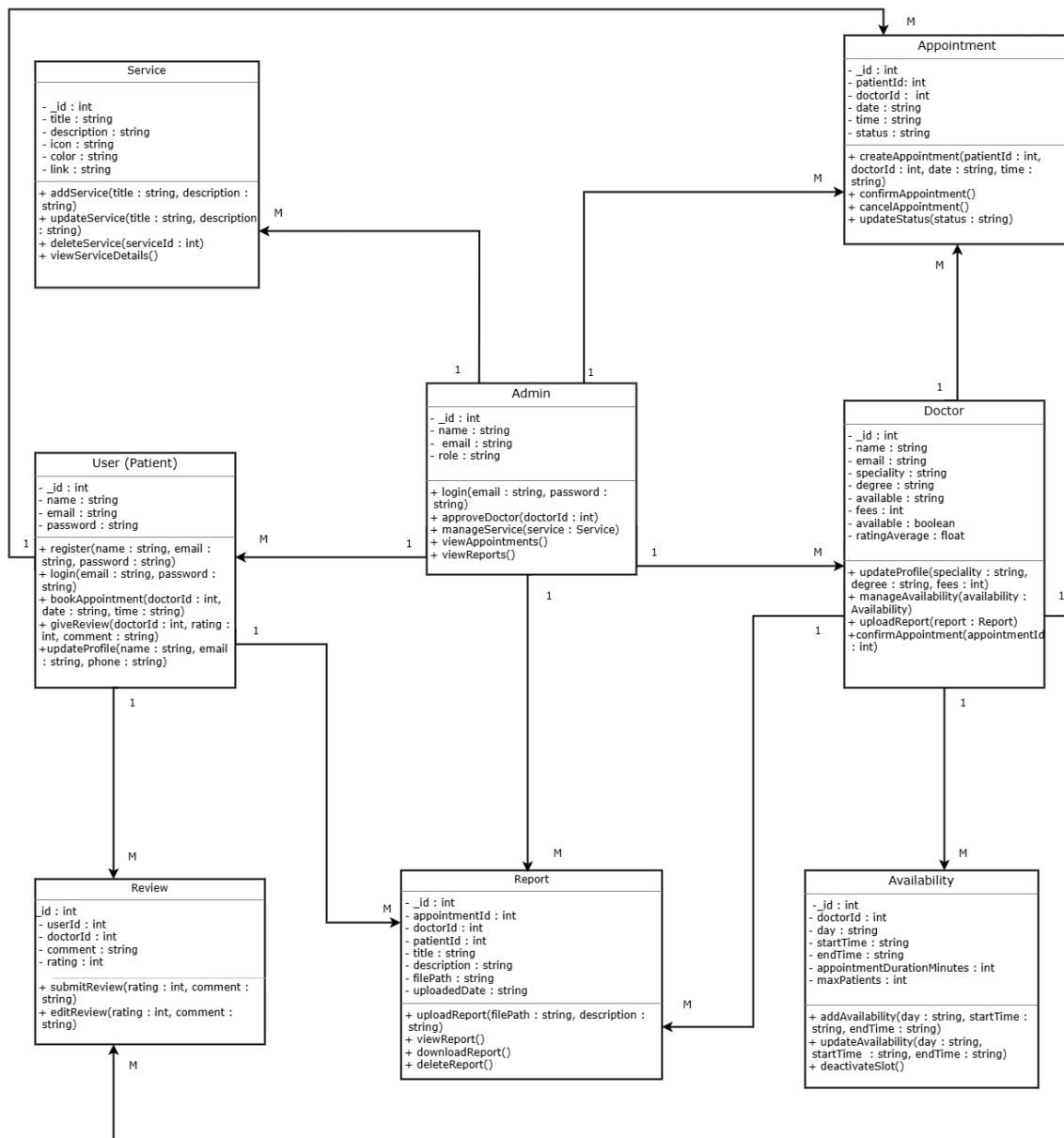


Figure 3.2: Class diagram for MediVision

The MediVision class diagram models the main components of the system and their interactions. The User/Patient class stores basic details and allows patients to book appointments, view reports, and give reviews. The Doctor class contains professional information, manages availability, appointments, and uploads reports. The Appointment class links patients and doctors, storing details like date, time and status. Availability defines doctors' working schedules, while the Report class stores medical reports for patients. The Review class connects patients and doctors through ratings and feedback and the Services class are managed by the admin. Together, these classes illustrate the overall structure and flow of the MediVision system.

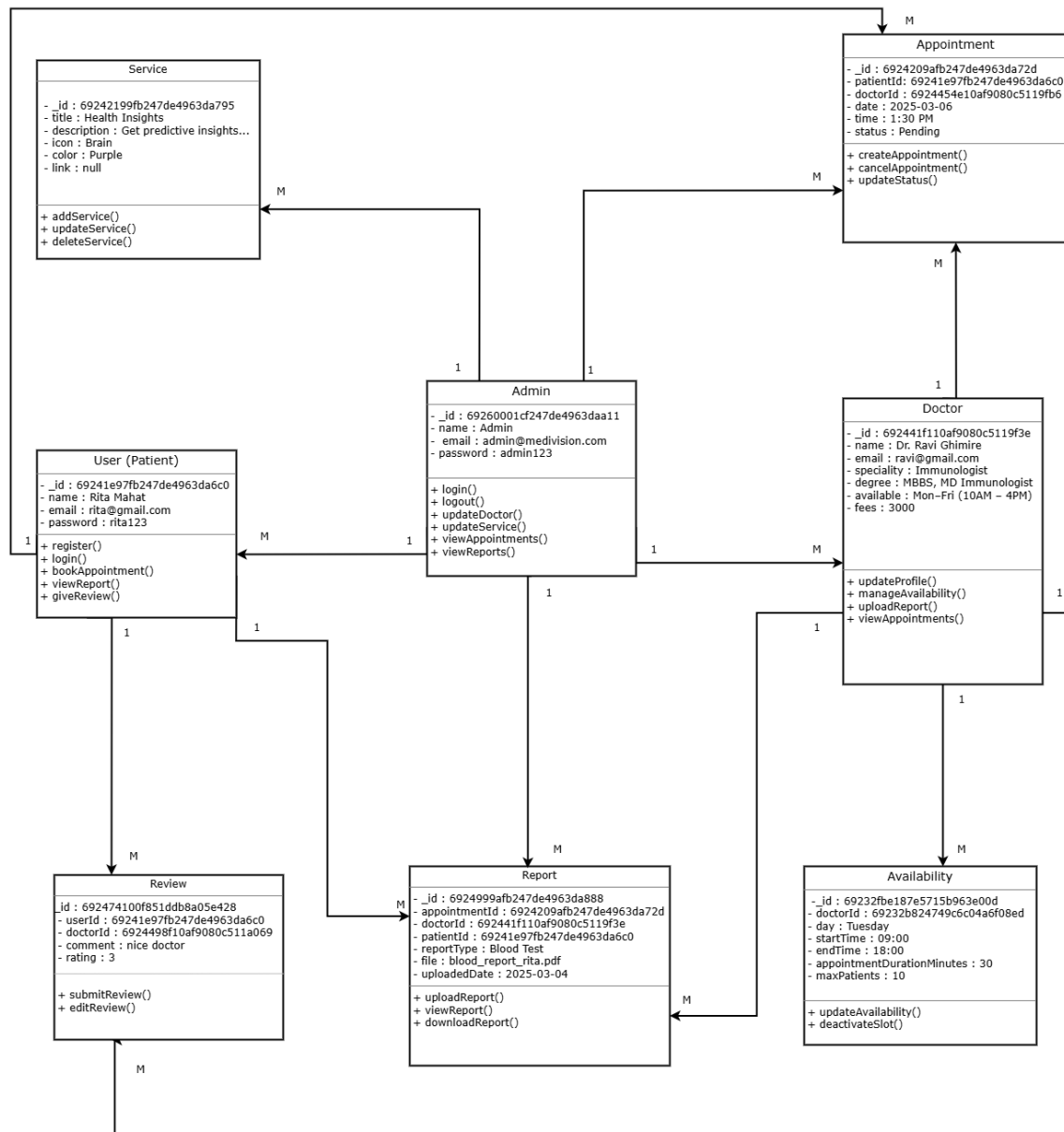


Figure 3.3: Object diagram for MediVision

The MediVision object model represents the real instances of system entities and their interactions. The User object stores patient details and enables appointments, reports, and reviews. The Doctor object holds professional information, manages availability, appointments, and uploads reports. The Appointment object links patients and doctors with date, time and status. The Availability object defines doctors' schedules, while the Report and Review objects store medical reports and patient feedback. The Service object represents platform features managed by the admin. Together, these objects show how the MediVision system operates with actual data.

3.1.3 Dynamic Modeling using sequence diagram

Dynamic modeling in MediVision shows how different users like patients, doctors, and administrators interact with the system over time. It explains how patients can enter their symptoms, get a health prediction, and then book an appointment with the right doctor based on the result. The model helps show the flow of activities such as booking, updating doctor schedules, and managing patient records. Overall, it ensures that the system runs smoothly, handles data efficiently, and provides an easy and smart way for users to manage healthcare services.

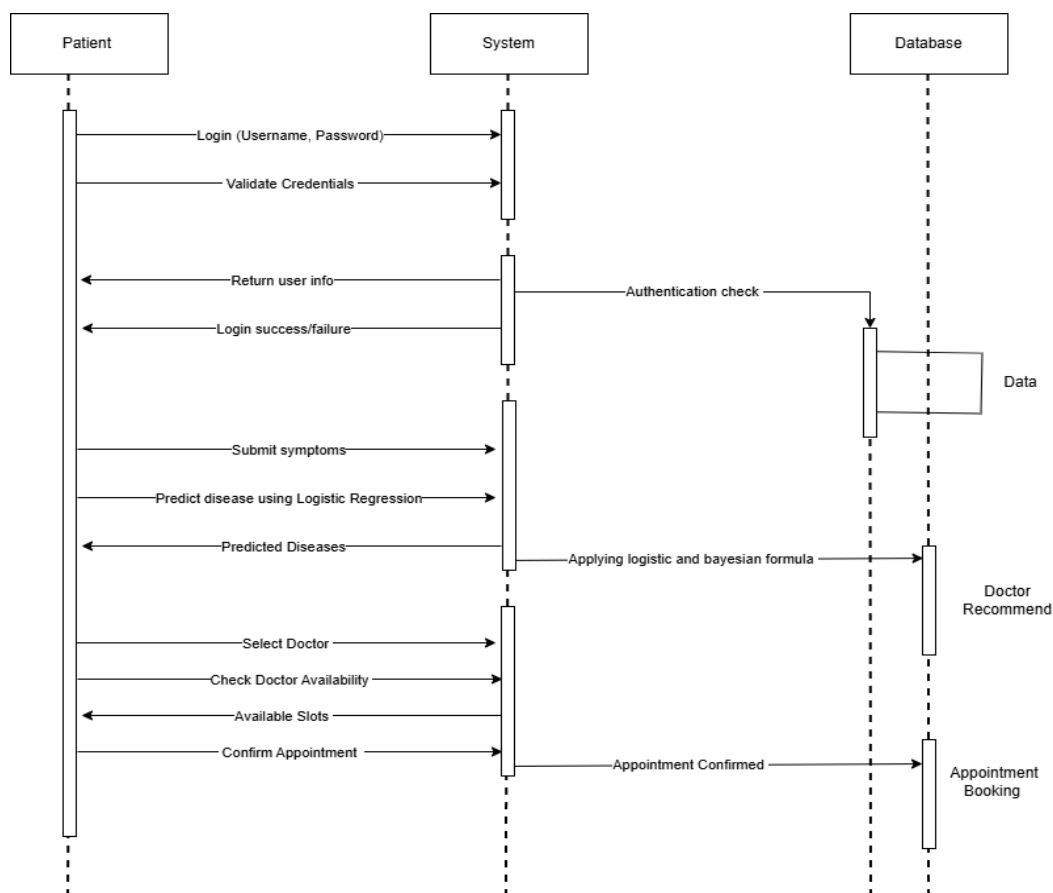


Figure 3.4: Sequence diagram for MediVision

This sequence diagram illustrates the process of a patient interacting with the MediVision system, starting with login, where the system authenticates the patient's credentials via the database and confirms successful access. The patient then enters their symptoms, which the system processes to generate a list of possible diseases. Next, the patient selects a doctor, and the system checks the doctor's availability through the database, presenting the available slots. Finally, the patient confirms the appointment, and the system records it in the database and sends a confirmation back to the patient, ensuring a seamless flow from symptom entry to appointment booking.

3.2.3 Process Modeling using Activity diagram

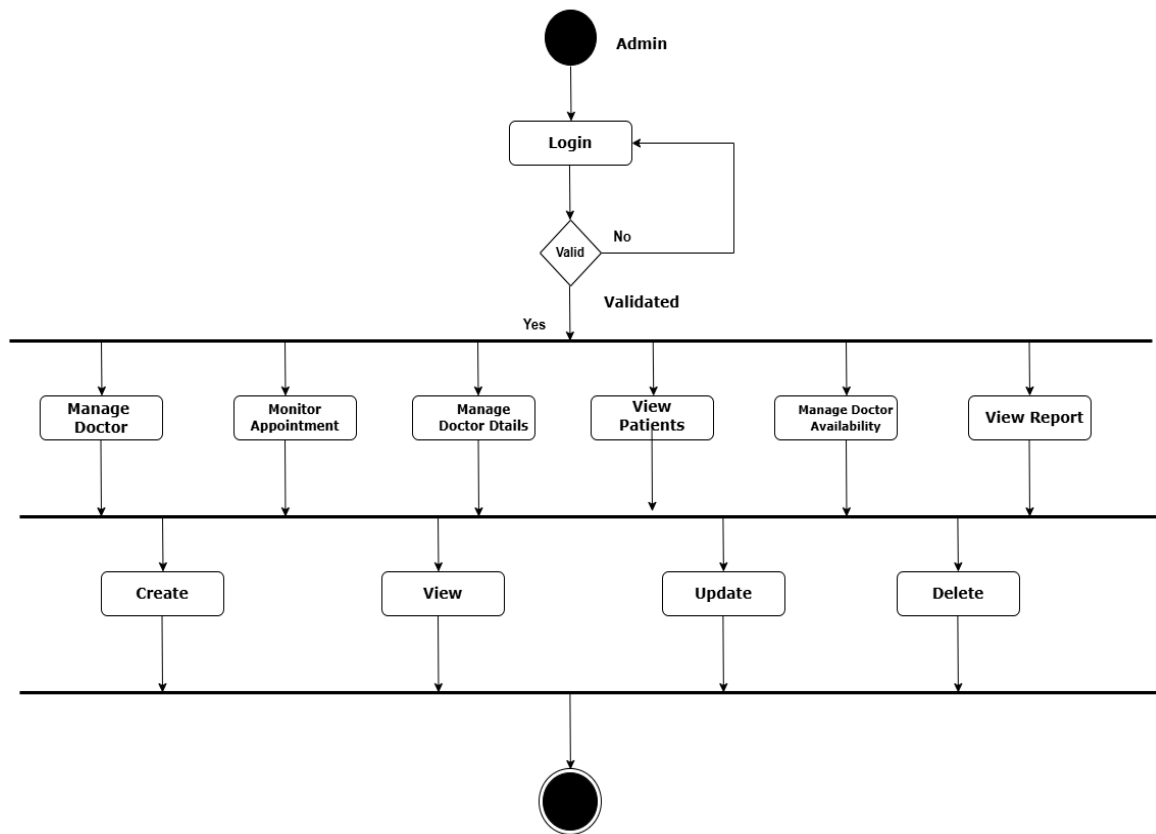


Figure 3.5: Activity diagram for Admin for MediVision

The activity diagram starts with admin login and validation. If successful, the admin can manage doctors, view users, monitor appointments, and view reports. If the login fails, an error message is shown and the admin returns to the login page. The process ends with logout.

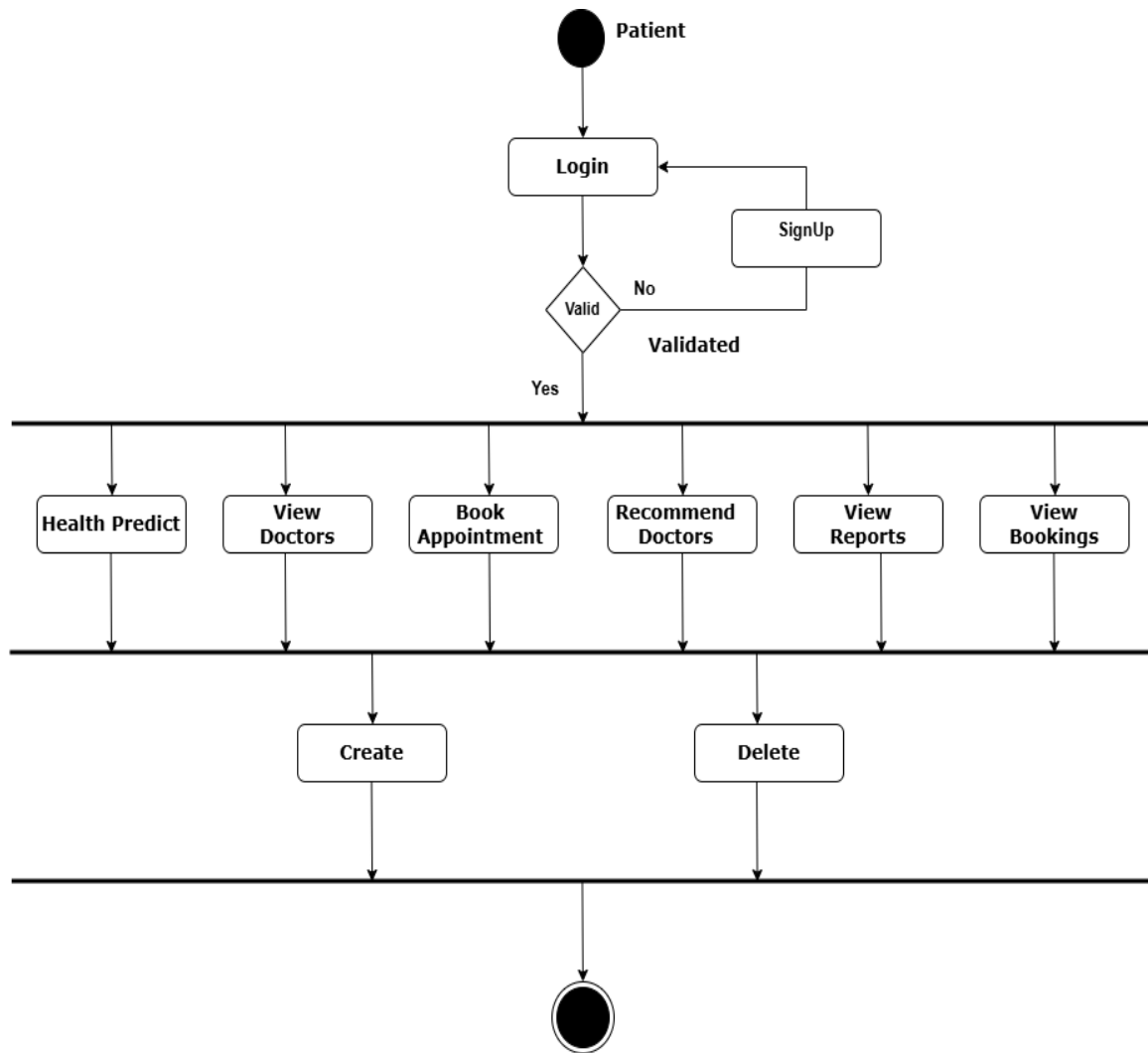


Figure 3.6: Activity diagram for Patient for MediVision

The activity diagram starts with patient login and validation. If successful, the patient can enter symptoms, view disease prediction, search for doctors, book appointments, and view reports or appointment status. If login fails, an error message is shown and the patient is returned to the login page. The process ends with logout.

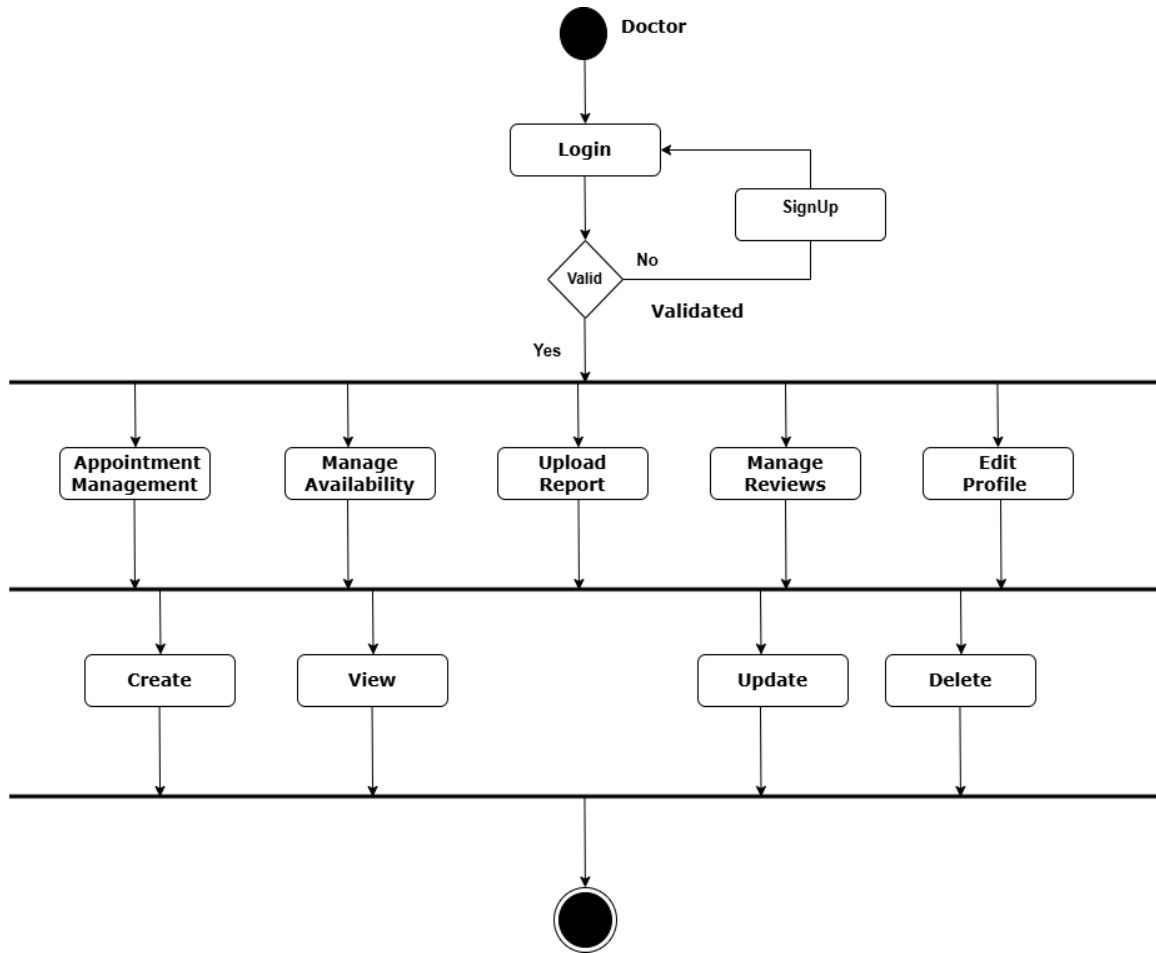


Figure 3.7: Activity diagram for Doctor for MediVision

The activity diagram begins with doctor login and validation. If successful, the doctor can manage profile and schedule, view and respond to appointment requests, consult with patients, and update reports or prescriptions. If login fails, an error message is displayed and the doctor is returned to the login page. The process ends with logout.

3.2 System Design

To clearly represent all the system requirements in a graphical way, various diagrammatic designs of the system were prepared, which are as follows:

3.2.1 Refinement of Class diagram

Refining the class diagram means enhancing it with additional details and making the relationships between classes clearer. This process elaborates on the initial design to give a more complete and accurate representation of the system. Below is the refined version of the previously created class and object diagrams for the system:

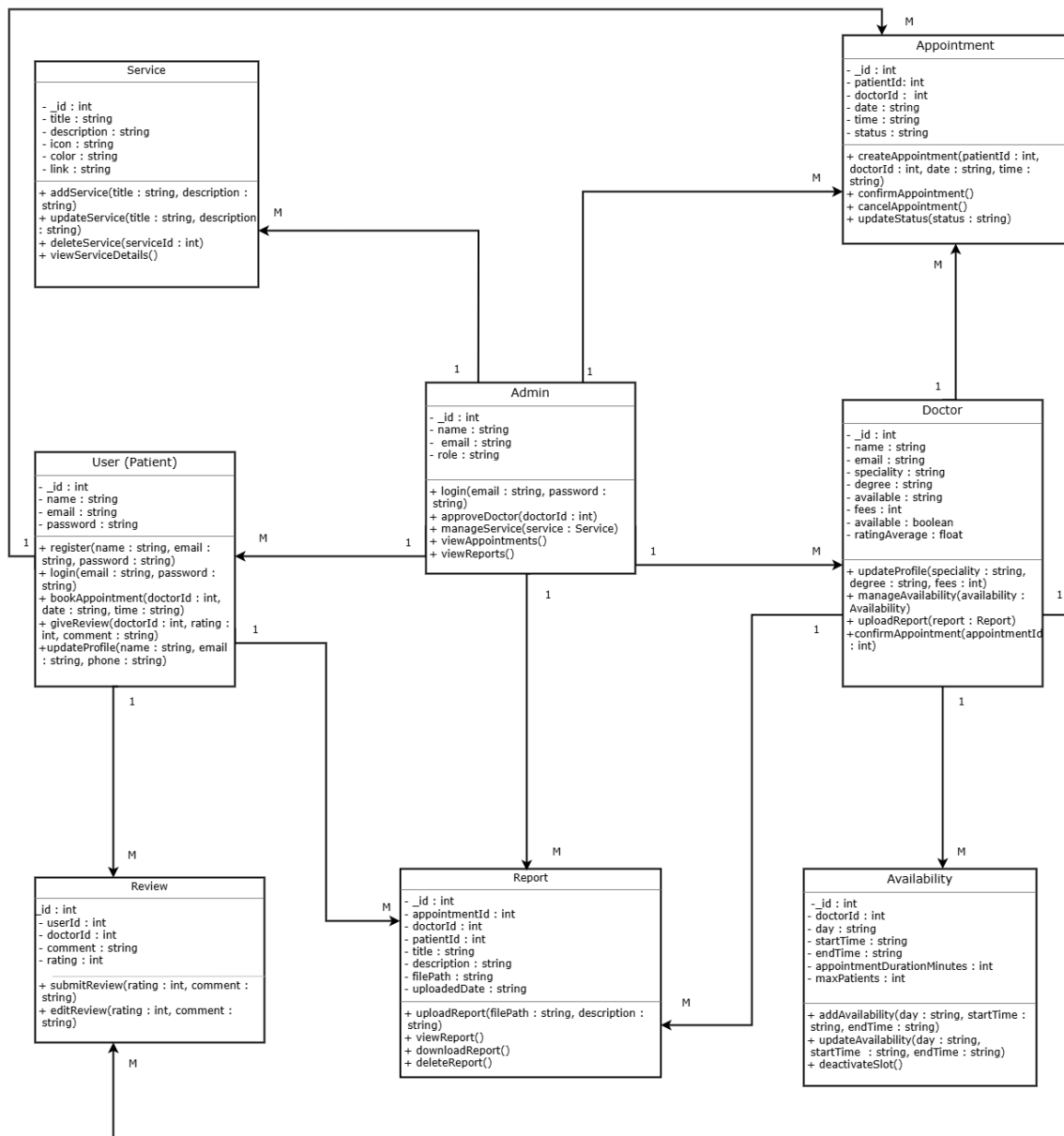


Figure 3.8: Refinement class diagram of MediVision

Refining the class diagram means enhancing it with additional details and making the relationships between classes clearer. This process elaborates on the initial design to give a more complete and accurate representation of the system. Above is the refined version of the previously created class and object diagrams for the system.

3.2.2 Component Diagram

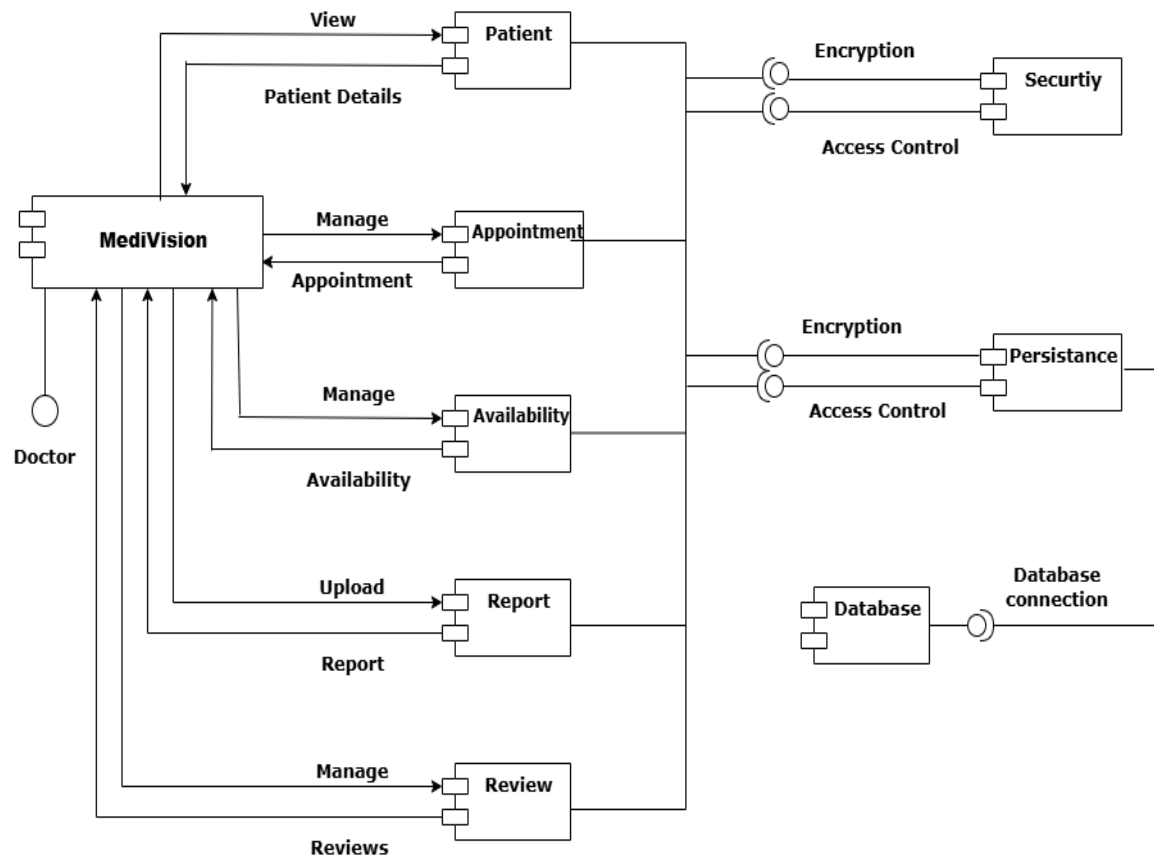


Figure 3.9: Component diagram for Doctor of MediVision

The diagram represents the structure of an object-oriented system for the Doctor module in MediVision, which is designed to manage various aspects of doctor activities and patient interactions. At the core is the Doctor component, which connects with Appointment to handle booking requests, Patient (User) to view medical details, and Report to create and update diagnosis and prescriptions. The Schedule/Availability component manages working hours, while Profile stores personal and professional information.

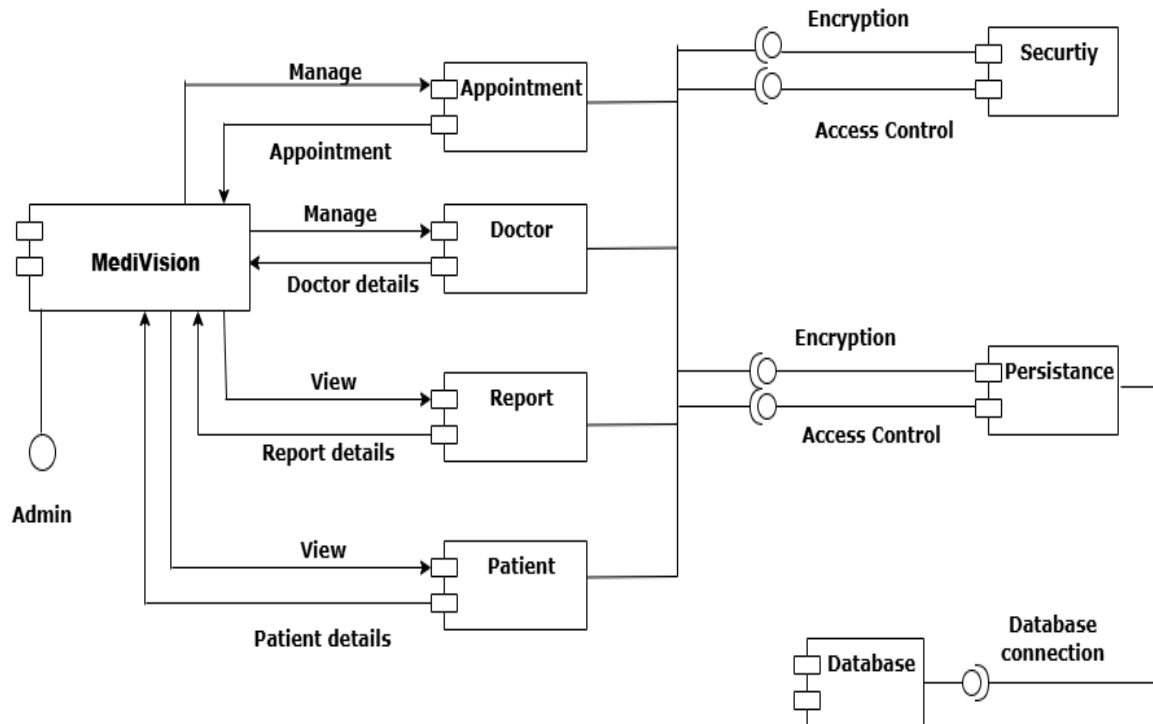


Figure 3.10: Component diagram for Admin of MediVision

The diagram shows the structure of the Admin module in MediVision. At the center is the Admin component, which is connected to Doctor (to manage and approve doctors), Patient/User (to view patient information), Appointment (to view and manage bookings), and Report (to view and generate system reports). The Dashboard allows the admin to access and control all these components efficiently.

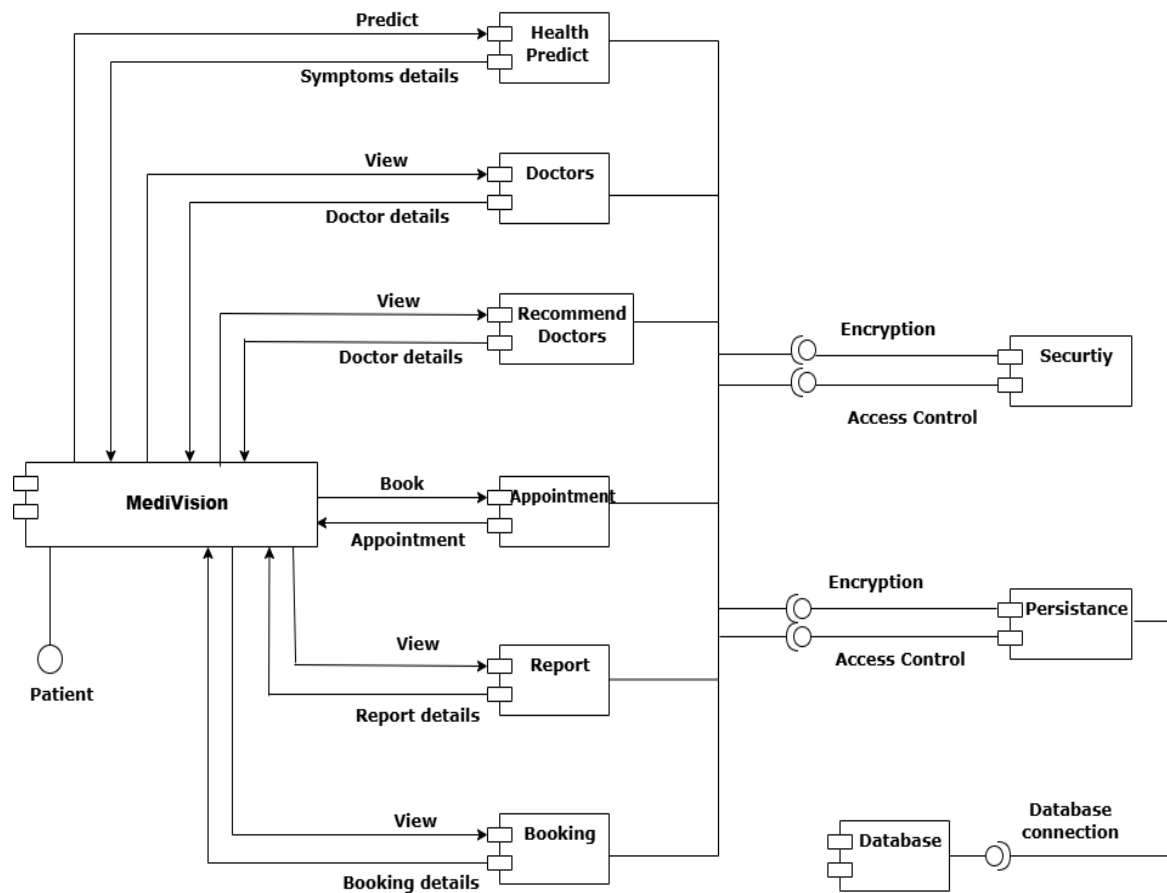


Figure 3.11: Component diagram for Patient of MediVision

The diagram represents the structure of the Patient module in MediVision, which is designed to support patient interactions with the system. At the core is the Patient (User) component, which connects with doctor to view doctor information, Appointment to book and track appointments, and Report to view medical reports and predictions. The Profile component stores personal details, while the Symptom/Input component allows patients to enter health data for analysis.

3.2.3 Deployment Diagram

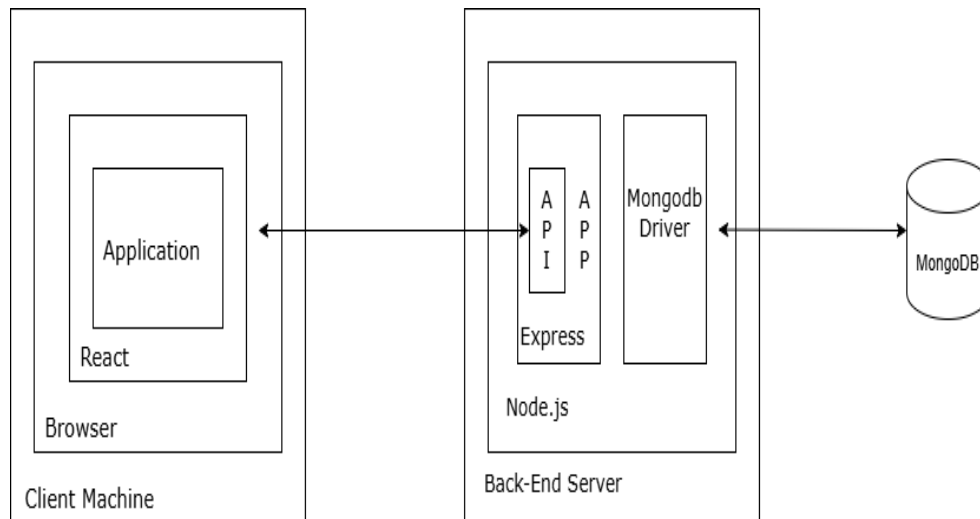


Figure 3.12: Deployment diagram of MediVision

The diagram shows a basic MediVision system architecture where a client machine (user's device) interacts with the system through a web browser. The browser loads the React application, which acts as the user interface for patients, doctors, and admins. On the other side, the backend server uses Node.js and Express to process requests coming from the React application through APIs. These APIs handle all system operations such as login, symptom analysis, appointment booking, and report management. The MongoDB driver connects the backend to the MongoDB server, where all user, doctor, appointment, and report data is securely stored and retrieved. This deployment setup allows smooth communication between the front end and the database through the backend services.

3.2.4 Wireframing

Wireframing is like creating a simple blueprint of a website or application before actual design and development begin. It helps to decide where different elements will be placed and how users will navigate through the system.

In the MediVision system, wireframing helps plan the layout of important pages such as:

- **Patient portal** where users can enter symptoms, view predictions, and access appointment history.
- **Doctor dashboard** where doctors can manage their profile, set availability, upload report and view patient appointments.
- **Admin panel** where administrators can manage doctors, view patients, monitor appointments, and generate reports.

Wireframes visually represent key elements like login forms, symptom input fields, doctor lists, appointment booking buttons, and report sections. This ensures the MediVision system is well-organized, easy to use, and efficient before moving to the design and implementation stage.

Homepage:

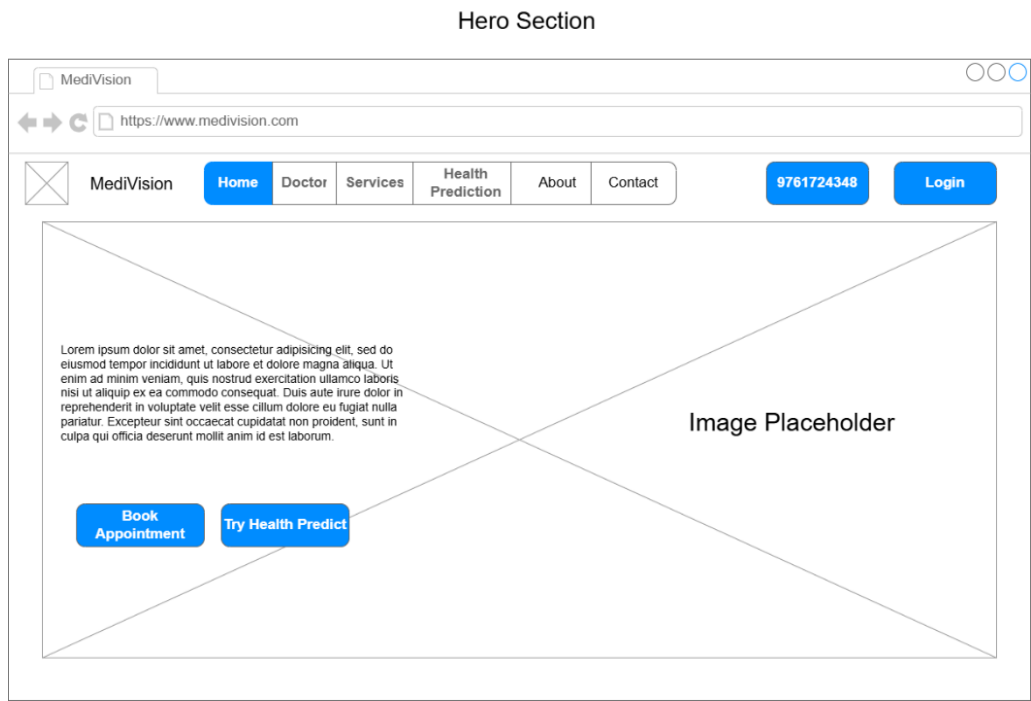


Figure 3.13: Wireframing Design of Homepage

This shows the MediVision homepage with a navigation bar, logo, login/signup buttons, a brief description, and a main button like “Book Appointment” and “Try Health Predict” along with a medical-themed image or slider.

User Registration Page:

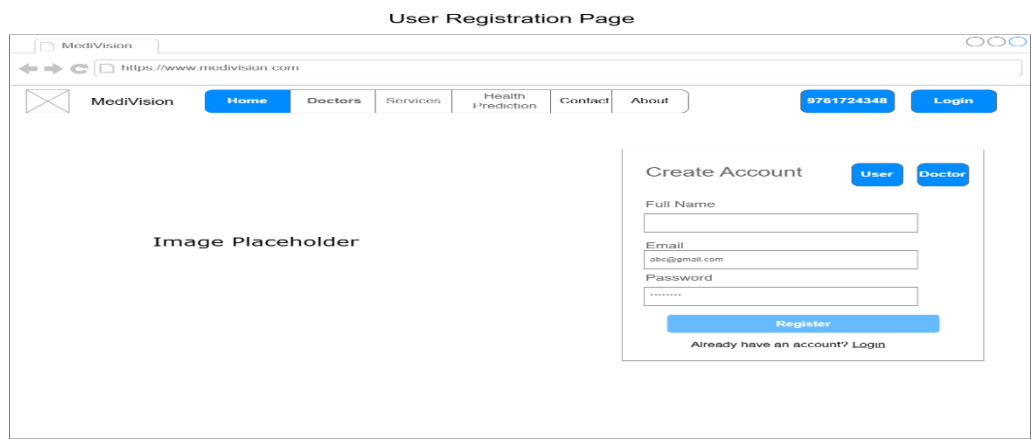


Figure 3.14: Wireframing Design of User Registration Page

This shows a patient registration form with fields for basic details such as name, email and password, and a register button.

Doctor Dashboard:

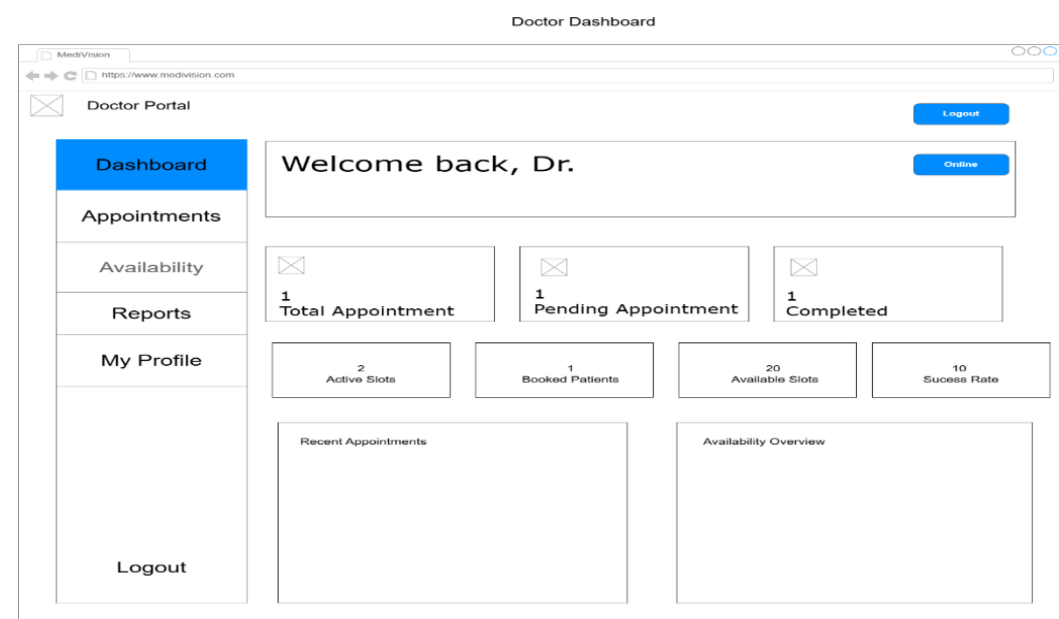


Figure 3.15: Wireframing Design of Doctor Dashboard

This shows a doctor’s panel displaying appointments, availability, report uploads, and profile management, along with basic statistics.

Admin Dashboard:

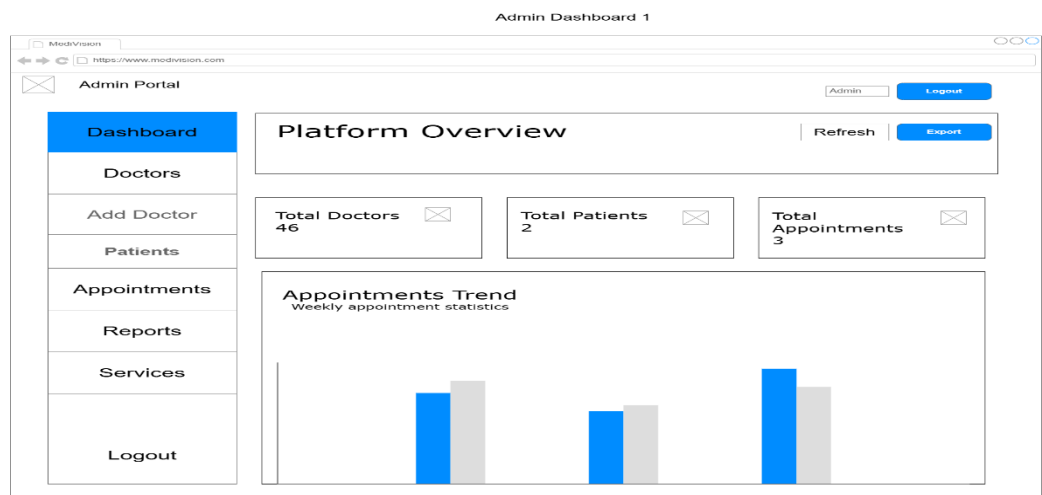


Figure 3.16: Wireframing Design of Admin Dashboard

This shows the admin panel for managing patients, doctors, appointments, and reports, with summary charts for system monitoring.

3.2.5 Interface Design (UI Interface/Interface Structure Diagrams)

The process of planning and arranging the visual layout and interactive elements of a software or app to ensure a clear and user-friendly experience.

Home page:

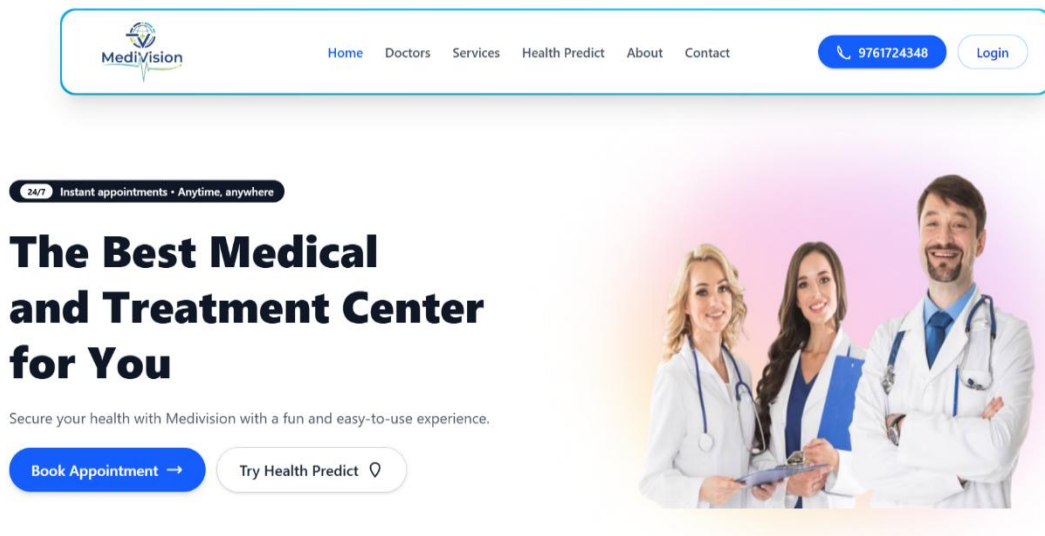


Figure 3.17: Interface Design of Homepage

This is the homepage of MediVision, showing a simple and user-friendly interface that allows users to access health services such as symptom checking, disease prediction, and doctor appointment booking.

Health Prediction:

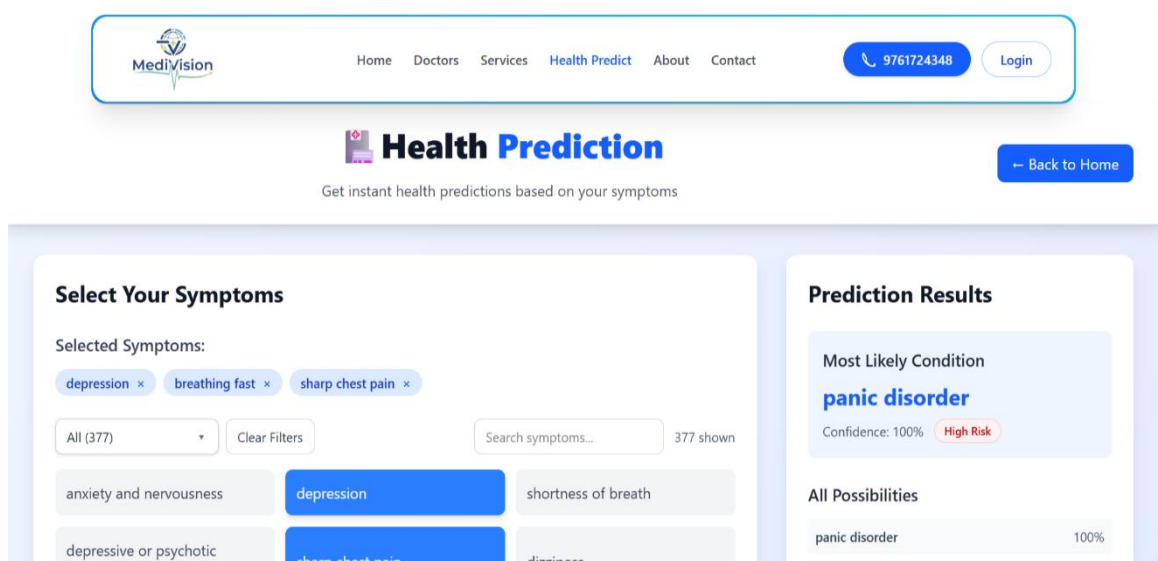


Figure 3.18: Interface Design of Health Predict Page

It is a feature that allows patients to enter their symptoms and receive a possible disease prediction using the system's intelligent model. This helps users understand their health condition before consulting a doctor.

Appointment Booking:

The screenshot shows the Medivision Appointment Booking Page. At the top is a navigation bar with the Medivision logo, links for Home, Doctors, Services, Health Predict, About, and Contact, a phone number 9761724348, and a user profile for Rita mahat. The main content area is divided into two columns. The left column features a doctor's profile for Dr. Ruby K.C., an MBBS, MD Cardiologist with 5 years of experience. It includes a consultation fee of Rs. 5000 and a status of 'Available'. A bio states: 'I am a dedicated cardiologist focused on diagnosing, treating, and preventing heart-related conditions. I work closely with patients to monitor heart health, recommend effective treatments, and promote healthy lifestyles with a compassionate and detail-oriented approach.' The right column has a 'Choose a Day' section with buttons for SAT (29, 16 slots) and SUN (30, 16 slots). Below this is a 'Choose a Time' section with a grid of time slots. The 10:10 AM slot is selected. At the bottom, an 'Appointment Summary' section shows 'Selected: Saturday, November 29, 2025 at 10:10 AM' and a 'Book Appointment' button.

Figure 3.19: Interface Design of Appointment Booking Page

It is the appointment booking page where patients can search for doctors, select available time slots, and confirm appointments for consultation and treatment.

3.3 Algorithm Details

3.3.1 Logistic Regression Algorithm

In Medivision, logistic regression is used to predict health conditions of patients based on their input data, such as symptoms, age, and medical history. Since the prediction is a binary outcome (e.g., disease present or not), logistic regression is ideal because it calculates the probability of a patient belonging to a specific class (Yes/No, 0/1). Its ability to handle classification problems makes it suitable for generating meaningful health predictions that can guide doctors and patients. [15]

Logistic regression can be classified into three main types based on the nature of the dependent variable:

- **Binomial Logistic Regression:** Used when the outcome has only two possible categories, such as predicting whether a patient has a particular health condition or not (Yes/No, 0/1). This is the most relevant type for Medivision's health prediction feature.

- **Multinomial Logistic Regression:** Used when the outcome has three or more categories without any order, for example, predicting different types of conditions or diseases a patient might have. This can be applied if MediVision expands to classify multiple disease types at once.
- **Ordinal Logistic Regression:** Used when the outcome has three or more categories with a natural order, such as severity levels of a condition (mild, moderate, severe). This can help MediVision prioritize patients based on the severity of their predicted health condition.

Assumptions of Logistic Regression in MediVision

To ensure accurate health predictions, logistic regression in MediVision follows these assumptions:

- **Independent observations:** Each patient's data is treated independently, so one patient's information does not affect another's prediction.
- **Binary dependent variables:** The primary prediction is whether a patient has a health condition or not (Yes/No, 0/1). For multiple disease categories, multinomial approaches can be used.
- **Linearity relationship between predictors and log odds:** Features like symptoms, age, and test results are assumed to affect the log odds of the outcome in a linear way.
- **No extreme outliers:** Data should not have extreme or unusual values, as they can distort prediction accuracy.
- **Sufficient sample size:** A large number of patient records ensures stable and reliable predictions.

Understanding the Sigmoid Function in MediVision

In MediVision, the sigmoid function is used to convert the raw output from the logistic regression model into a probability between 0 and 1, indicating the likelihood of a patient having a health condition. It forms an "S"-shaped curve called the sigmoid curve, which is ideal because probabilities must be between 0 and 1.

A threshold value (usually 0.5) is used to decide the class:

- $\text{Probability} \geq 0.5 \rightarrow$ Patient is predicted to have the condition (Class 1)
- $\text{Probability} < 0.5 \rightarrow$ Patient is predicted not to have the condition (Class 0)

This allows continuous input features like symptoms, age, and test results to be transformed into meaningful health predictions for the system.

How does Logistic Regression work?

In Medivision, logistic regression transforms the continuous output of a linear combination of patient features (like symptoms, age, and test results) into a categorical prediction using the sigmoid (logistic) function.

- Input features are represented as a vector $\mathbf{X}$, with corresponding weights $\mathbf{w}$ and a bias $\mathbf{b}$. The linear function calculates $z = \mathbf{w} \cdot \mathbf{X} + \mathbf{b}$, which is a continuous value.
- The sigmoid function converts this z value into a probability between 0 and 1, showing the likelihood of a patient having a condition.
- Using a threshold (usually 0.5):
 - Probability $\geq 0.5 \rightarrow$ Class 1 (condition present)
 - Probability $< 0.5 \rightarrow$ Class 0 (condition absent)

The sigmoid function ensures that:

- $\sigma(z) \rightarrow 1$ as $z \rightarrow \infty$
- $\sigma(z) \rightarrow 0$ as $z \rightarrow -\infty$
- $\sigma(z)$ is always between 0 and 1

This allows Medivision to convert continuous patient data into meaningful health predictions, which can guide doctors in decision-making.

3.3.2 Bayesian Ranking Algorithm

In Medivision, The Bayesian Ranking Algorithm ranks doctors by combining their average rating (R) and number of reviews (V) with a minimum review threshold (m) and specialty-group average rating (globalMean).

It prevents doctors with very few reviews from being overrated and ensures fair, stable ranking.

The formula used is:

$$\text{Score} = (V/(V+m)) * R + (m/(V+m)) * \text{globalMean}$$

This means the doctor's score depends partly on their own ratings and partly on the average rating of all doctors in the same specialization group.

Implementation of algorithm:

1. Page Method (page function):

Handles user requests and retrieves inputs like:

- patient symptoms
- preferred specialization

- doctor availability

The system also retrieves each doctor's:

- average rating (R)
- number of reviews (V)
- specialty group average rating (globalMean)

These inputs are sent to the view along with the doctor data for Bayesian scoring.

2. **CleanText Method (cleanText function):**

Cleans input text (removes stop words, punctuation, converts to lowercase) if symptoms or descriptions are used.

Text cleaning here is mainly for filtering doctors by specialization or condition, not for similarity scoring.

3. **Algorithm Method (algorithm function):**

This method performs the doctor recommendation using the Bayesian rating formula.

- **Step 1: Exact Match** - The system first retrieves doctor profiles that exactly match the user's selected i.e. specialization and availability.

These doctors are included directly in the result list, but their ranking is still based on the Bayesian score.

- **Step 2: Rating-Based Ranking (Using Bayesian Ranking Algorithm)** – For every doctor (matched or unmatched), the system calculates a weighted rating using i.e. Doctor's average rating (R), Number of user reviews (V), Minimum required review threshold (prior strength) (m) and Average rating of all doctors in the same specialization (globalMean).

The formula:

$$\text{Score} = (V/(V+m)) * R + (m/(V+m)) * \text{globalMean}$$

Explanation:

- When a doctor has many reviews (V), their own rating R dominates.
- When a doctor has few reviews, the score shifts toward globalMean, preventing unfair inflation.
- m controls how strongly the global specialty rating influences the score.

The final output in MediVision consists of:

- Doctors who exactly match the user's filters (specialization or availability).
- Doctors ranked based on their Bayesian weighted rating score.

The system:

- Combines exact matches and Bayesian-scored matches.
- Removes duplicate doctor IDs.
- Sorts the final list by Score in descending order.

This ensures the top doctors are those with reliable, trustworthy ratings.

The Bayesian Ranking Algorithm combines exact doctor matching with a reliable rating-based scoring system. It uses a weighted formula that accounts for each doctor's average rating, the number of reviews, and the overall specialty group's rating, preventing bias from low-review doctors and ensuring fair, trustworthy recommendations in MediVision.

Chapter 4: Implementation and Testing

4.1 Implementation

4.1.1 Tools Used

Various tools were implemented for the development of this project. Following are the tools that are used for the development of this projects:

Frontend Tools

ReactJS: In the MediVision system, ReactJS is used for building the interactive user interface of the application. React's component-based architecture allows the system to render reusable UI elements for pages like patient dashboards, doctor panels, appointment forms, and prediction results. With React Hooks and state management, the application becomes more dynamic, responsive, and efficient.

HTML5: HTML5 is used for structuring the overall layout of the MediVision web pages. All elements such as buttons, headings, paragraphs, forms, images, and links are created using HTML. It provides the semantic structure upon which React components are built.

CSS3: CSS3 is used to style all of the HTML and React components, ensuring a clean, modern, and consistent visual appearance. CSS controls colors, layout design, font styles, animations, spacing, and responsiveness of the application. It enhances the user experience by making the interface visually appealing and easy to navigate.

Backend Tools

Node.js: Node.js is used as the server-side runtime environment for MediVision. It enables fast, scalable, and efficient handling of API requests between the frontend and the backend. Using Node's event-driven architecture, the system can process tasks like authentication, appointment scheduling, and report handling smoothly.

Express.js: Express.js is used as the backend framework for building RESTful APIs. It handles routing, middleware, request validation, and backend logic for different modules like doctors, patients, reports, and appointments. Express simplifies server development and improves code maintainability across the project.

Database

MongoDB: MongoDB is used as the main database for storing all information related to patients, doctors, medical reports, appointments, and prediction records. MongoDB's flexible document-based structure allows efficient storage of dynamic health-related data.

CRUD operations: Create, Read, Update, and Delete are implemented through Mongoose, which simplifies schema creation and ensures proper validation of data across the system.

Documentation Tools used

MS Word: MS word is one of the powerful tools for creating, editing, formatting, and printing documents. Here, in this project MS Word was used for the documentation of the project which contains the function like page layout, tables and columns, templates, images and so on.

Draw.io: Draw.io is a popular one diagramming and flowcharting tool. Using its interface all of the diagrams were created for the system. All of the diagrams were created using this tool. We don't need to install any of the software for accessing it.

4.1.2 Implementation Details of Modules

The system modules are shown below:

- **User Registration Module**

```
const { name, email, password } = req.body
if (!name || !email || !password) {
  return res.status(400).json({ message: 'Name, email and password are required' })
}
```

Listing 4.1: Code Snippet for Patient Registration

This module checks whether the user has entered their name, email, and password correctly before moving forward. If any of these fields is empty or missing, the system immediately stops the process and shows an error message, making sure that incomplete or invalid information is not accepted by the server.

- **User Login Module**

```
const { email, password } = req.body
if (!email || !password) {
  return res.status(400).json({ message: 'Email and password are required' })
}

const user = await User.findOne({ email })
if (!user) {
  return res.status(401).json({ message: 'Invalid credentials' })
}
```

Listing 4.2: Code Snippet for Patient Login

This module first checks if the user has provided both email and password in the login request. If either one is missing, it immediately sends a 400-error saying that both fields are required. After that, the system searches the database for a user with the entered email. If no user is found, it returns a 401 error (Invalid credentials), preventing unauthorized access and making the login process more secure.

- **Bayesian Ranking Module**

```
export function computeBayesianScores(items, {
  groupKey = (d) => (d.speciality || 'ALL').toLowerCase(),
  ratingGetter = (d) => Number(d.ratingAverage ?? d.rating ?? 0) || 0,
  countGetter = (d) => Number(d.ratingCount ?? d.reviewsCount ?? d.numReviews ?? 0) || 0,
  priorStrength = 5,
} = {}) {
  if (!Array.isArray(items) || items.length === 0) return [];
}
```

Listing 4.3: Code Snippet for Bayesian Ranking Algorithm

This module defines a function called `computeBayesianScores` that is used to calculate fair and balanced scores for a list of items (such as doctors or courses). It groups the items by specialty, collects their ratings and number of reviews, and then uses a Bayesian method to avoid giving too much importance to items with very few reviews.

First, it checks if the input items are valid array and not empty. If there is no data, it simply returns an empty list. This helps prevent errors and makes sure the system only works with valid and meaningful data.

- **Logistic Regression Module**

```
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
```

Listing 4.4: Code Snippet for Logistic Regression Algorithm

Logistic Regression model using the given training data, then use the trained model to predict the diseases from the test data, and finally calculate the accuracy to check how correctly the model is identifying diseases based on the symptoms.

- **Appointment Booking Module**

```
export const getAppointmentDetails = async (req, res) => {
  try {
    const { id } = req.params
    const a = await Appointment.findById(id).lean()
    if (!a) return res.status(404).json({ message: 'Appointment not found' })
    const doctor = a.doctorId ? await doctorModel.findById(a.doctorId).select('name image speciality').lean() : null
  }
}
```

Listing 4.5: Code Snippet to Book an Appointment

This module creates a new appointment using the patient's input data (such as name, doctor, date, and time) and saves it in the database. Once saved, the appointment is stored with a pending status and can later be approved or rejected by the admin or doctor.

- **Patient Report Module**

```
const reports = await Report.find(query)
  .sort({ createdAt: -1 })
  .select('-filePath')
  .lean()

return res.status(200).json({
  reports,
  message: 'Reports retrieved successfully'
})
```

Listing 4.6: Code Snippet for Patient Report Management

These modules fetch all reports from the database that match the given criteria, sort them so the newest reports appear first, exclude the file paths for security, and then send the list of reports along with a success message back to the user.

For Admin,

- **Admin Login:**

Admin can log into the MediVision system using valid credentials. After logging in, the admin can access the full administrative dashboard and manage system-wide functionalities.

- **Admin Manages Doctors:**

Admin has the feature of managing all the registered doctors in the system. Admin can add, edit, update, approve, reject, or delete doctor accounts. Admin can also view detailed doctor information such as Name, Email, Specialization, Experience, Availability, and Status.

- **Admin Manages Patients:**

Admin can manage all the registered patients in the system. Admin can delete patient accounts but cannot edit patient details. Admin can view patient information including Name, Email, Contact, Age, Medical History, and Appointment Records.

- **Admin Manages Appointments:**

Admin can view and manage all appointments created within the system. Admin can approve, reject, update appointment statuses, and ensure proper allocation of doctor availability to maintain a smooth scheduling process.

For Doctor,

- **Doctor Login:**

Doctors can log into the MediVision system with their registered account credentials. After logging in, they gain access to the doctor dashboard and tools.

- **Doctor Manage Availability:**

Doctors can add, edit, update, and delete their availability slots. These slots will be visible to patients while booking appointments.

- **Doctor View Appointments:**

Doctors can view all incoming appointment requests. They can accept, reject, or update appointment statuses. They can also view patient details related to each appointment.

- **Doctor Manage Reports:**

Doctors can create, upload, edit, update, and delete patient medical reports. They can also attach test results, prescriptions, and diagnosis notes within the system.

- **Doctor View Patient Records:**

Doctors can view patient medical histories, previous diagnoses, uploaded reports, and appointment logs. However, they cannot modify patient personal details.

For Patients,

- **Patient Login:**

Patients can log into MediVision using their registered email and password. After logging in, they can access all features related to their appointments and medical records.

- **Patient Register:**

New patients can create an account by filling in required information such as Name, Email, Password, Age, Contact Number, Address, Symptoms, and Profile Image.

- **Patient View Doctors:**

Patients can view the list of available doctors along with their specialization, experience, availability schedule, and profile details.

- **Patient Book Appointment:**

Patients can easily book appointments with doctors based on availability. They can select time slots, view appointment deadlines, and review booking status.

- **Patient View & Download Medical Reports:**

Patients can view all reports uploaded by doctors, including prescriptions, diagnoses, and test results. They can download their reports for personal use.

4.2 Testing

Testing has been carried out through different testing cases. The testing of the overall system has determined that our objectives are met and some of the test cases are as follow:

4.2.1 Test Cases for Unit Testing

Table 4.1: Unit Testing for MediVision

S.No.	Test Case Description	Test Data	Expected Outcome	Result
1.	Patient did not enter email during registration	Name: Ruby k.c Email: Password: ruby123	Error: Please fill out this field	Pass
2.	Patient enters incorrect email or password during login	Email: ruby12@gmail.com Password: passssss	Error: Invalid Credentials	Pass
3.	Doctor enters incorrect email or password	Email: krishab@gmail.com Password: passssss	Error: Invalid Credentials	Pass
4.	Doctor does not upload availability slot	Date: Start Time: 10:00 AM End Time: 12:00 PM	Error: Please select a valid date	Pass
5.	Patient books appointment without selecting doctor	Doctor: Date: 2025-02-10 Time: 1:00 PM	Error: Please select a doctor	Pass
6.	Doctor uploads medical report successfully	Patient: Ruby k.c Report: bloodreport.pdf Description: Normal condition	Report Uploaded Successfully	Pass
7.	Patient enters symptoms and gets prediction result	Symptoms: Fast breathing, chest pain Age: 22 Gender: Female	Prediction generated (e.g., Panic Discord)	Pass

4.2.2 Test Cases for System Testing

Table 4.2: System Testing for MediVision

S.No.	Test Case Description	Test Data	Expected Outcome	Result
1.	Patient views doctor list	User clicks "View Doctors"	Doctor list is displayed	Pass
2.	Admin deletes a patient record	Patient: Ruby k.c	Patient record is removed	Pass
3.	Admin approves doctor registration	Doctor: Dr. Milan (Pending)	Doctor account is approved	Pass
4.	Patient cancels an appointment	Appointment ID: 204567	Appointment is cancelled	Pass
5.	Doctor views patient's health prediction history	Patient: Ruby k.c	Prediction history is displayed	Pass
6.	Patient books an appointment successfully	Symptoms, Doctor, Date, Time	Appointment is confirmed	Pass
7.	Patient views uploaded medical reports	Patient logs into account	Reports are displayed	Pass
8.	Doctor updates personal profile information	Contact, Degree, Specialization	Profile is updated	Pass

Data Collection and Preparation

A dataset of 246,945 patient records was collected for disease prediction in the MediVision system. The dataset contains 607 different diseases and 377 unique symptoms, where each patient case represents the presence or absence of certain symptoms. The total number of values (cells) in the dataset is 93,345,210. The data was split into three sets: 70% for training, 15% for validation, and 15% for testing.

Table 4.3: Data Collection Test for MediVision

Parameter	Description
Total Records	246,945 patient records
Total Columns	378 columns
Total Values (Cells)	93,345,210
Number of Diseases (Classes)	607
Number of Symptoms (Features)	377
Primary Features	Symptoms (binary encoded)
Target Variable	Disease name (607 classes)
Training Set Size (70%)	~172,861 records
Validation Set Size (15%)	~37,042 records
Testing Set Size (15%)	~37,042 records
Data Type	Structured (Tabular Data)

Data Preprocessing

Several preprocessing steps were applied to improve the quality and performance of the data before training the model. These steps helped ensure consistency, reduce noise, and handle class imbalance between diseases.

Table 4.4: Data Preprocessing Test for MediVision

Stage	Task	Description
Data Cleaning	Missing Value Handling	Missing symptom values replaced using zero or mode
	Duplicate Removal	Duplicate patient records removed
	Outlier Detection	Abnormal values identified and reviewed
	Data Consistency	Ensured uniform binary format (0/1)
Feature Engineering	Label Encoding	Converted disease names into numeric values
	Binary Encoding	Converted symptoms into 0/1 format
	Normalization	Scaled values where required
	Feature Selection	Removed irrelevant or duplicated features
Data Balancing	Class Distribution Check	Identified imbalance in disease classes
	Class Weight Calculation	Class weights generated using sklearn
	Weight Assignment	Improved fairness in minority disease prediction

Model Training and Validation

After preprocessing the data, the Logistic Regression algorithm was selected for disease prediction because of its efficiency, simplicity, and strong performance on multi-class classification problems with large datasets. The model was trained using balanced class weights, L2 regularization, and optimized hyperparameters to improve generalization.

Table 4.5: Model Architecture for MediVision

Model	Algorithm	Input Size	Output	Regularization	Transfer Learning
Disease Prediction Model	Logistic Regression	377 features	607 classes	L2 Regularization	No

Model Testing Results

The model was evaluated using training, validation, and testing datasets. Performance was analyzed using accuracy, loss value, and top-3 accuracy.

Table 4.6: Model Training Test Results for MediVision

Model	Training Accuracy	Validation Accuracy	Training Loss	Validation Loss	Test Accuracy	Test Loss	Top-3 Accuracy
Logistic Regression	0.92 – 0.95	0.88 – 0.91	0.21 – 0.28	0.25 – 0.32	0.87– 0.90	0.30– 0.38	0.94 – 0.97

Test Set Evaluation

A detailed analysis was performed on the test dataset. The results show that the model performs well in predicting common and moderate diseases, while still maintaining acceptable accuracy for rare diseases.

Table 4.7: Disease Prediction Evaluation for MediVision

Evaluation Category	Metric	Logistic Regression
Overall Performance	Test Accuracy	~87 – 90%
	Top-3 Accuracy	~94 – 97%
	Test Loss	~0.30 – 0.38
	Confidence Threshold	20% (Basic), 80% (Improved)
Disease-Level Performance	Common Diseases	~90 – 93%
	Medium-Frequency Diseases	~86 – 89%
	Rare Diseases	~80 – 85%
System Efficiency	Average Prediction Time	< 2 seconds
	Scalability	Performs well with large datasets

4.3 Result Analysis

The system is designed to assist patients in finding the most suitable doctors and understanding their health conditions. Admins add doctor profiles with details, and patients receive recommendations based on ratings, availability, and their symptoms. The system uses algorithms like Bayesian Ranking for doctor recommendations and Logistic Regression for health predictions.

Top-Rated Doctor:

The system ranks doctors based on feedback, ratings, and appointment history to ensure fair and accurate ranking.

Workflow in the Algorithm:

- **Data Collection:** Gather all doctor ratings and feedback.
- **Prior Calculation:** Compute an initial baseline score to reduce bias for doctors with few reviews.
- **Bayesian Updating:** Adjust scores based on the number of ratings and their quality.
- **Ranking:** Doctors are sorted by their Bayesian score, with higher scores appearing at the top.

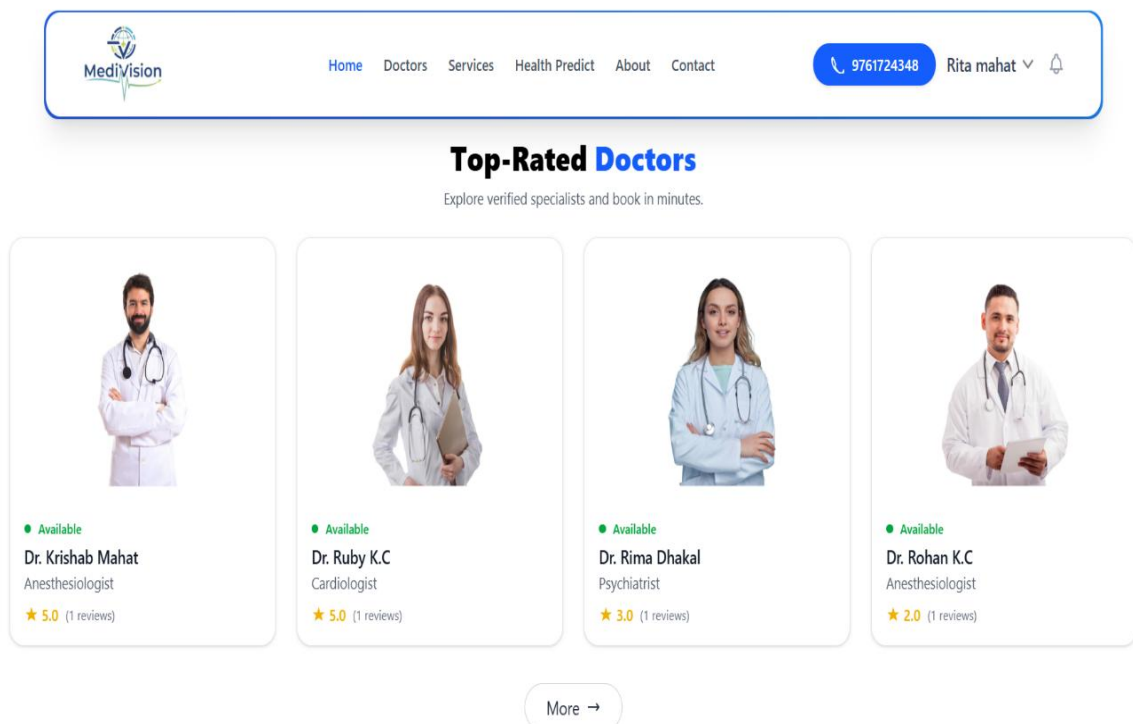


Figure 4.1: Top-Rated Doctor Using Bayesian Ranking

The system ranks doctors using feedback, ratings, and appointment history. It calculates a baseline score, updates it with Bayesian averaging based on review quality and quantity, and sorts doctors by their final scores.

Recommend Doctor:

Patients receive personalized doctor recommendations based on their requirements such as specialization, location, and availability.

Workflow in the Algorithm:

- **Patient Requirement Input:** Collect patient preferences and needs.
- **Doctor Matching:** Compare patient requirements with doctor profiles.

- **Bayesian Ranking:** Apply Bayesian algorithm to balance review scores and reliability.
- **Recommendation Output:** Display top-matched doctors to the patient.

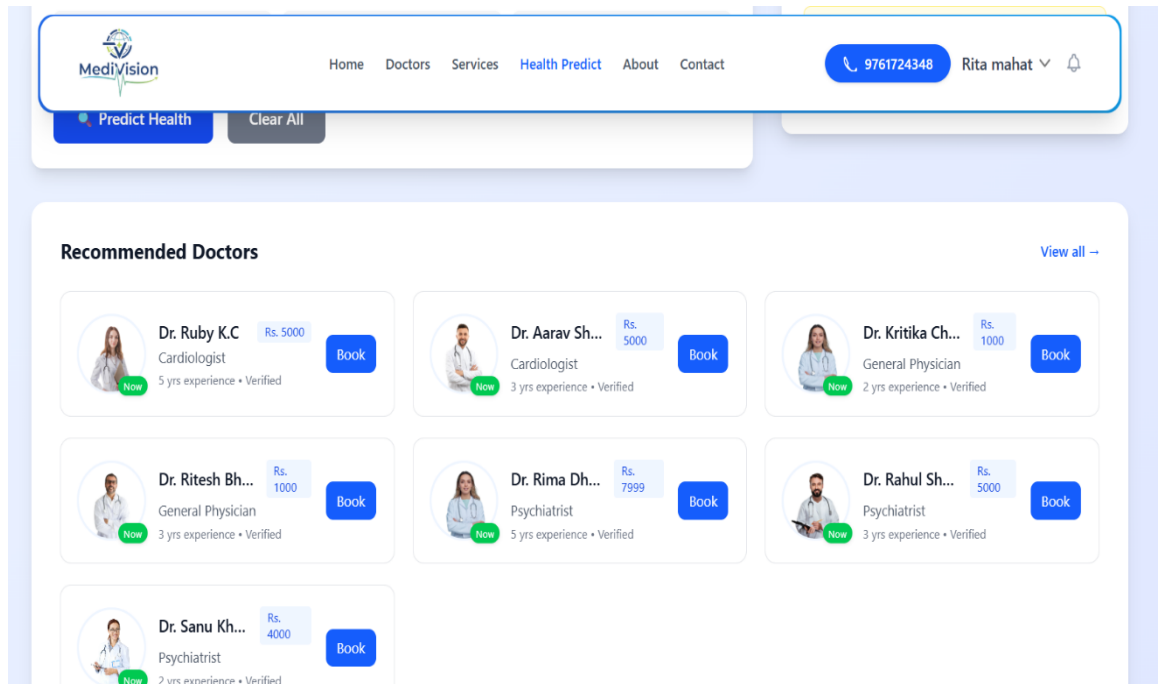


Figure 4.2: Recommended Doctors for Patient of MediVision

The system helps patients find the best doctors for their needs by considering specialization, location, and availability. It smartly matches patient preferences with doctor profiles, uses review scores to ensure reliability, and suggests the top doctors.

Health Prediction:

Patients enter symptoms into the system, which predicts possible diseases using logistic regression.

Workflow in the Algorithm:

- **Data Input:** Patients enter symptoms into the system.
- **Preprocessing:** Clean and encode symptom data for the model.
- **Prediction:** Logistic regression calculates the probability of possible diseases.
- **Result Output:** Display predicted conditions with likelihood scores for patient guidance.

The screenshot displays the MediVision Health Predict web application. The header includes the MediVision logo, navigation links (Home, Doctors, Services, Health Predict, About, Contact), a phone number (9761724348), and a user profile (Rita mahat). The main content area is divided into two sections: 'Select Your Symptoms' and 'Prediction Results'.

Select Your Symptoms:

Selected Symptoms: depression x sharp chest pain x irregular heartbeat x

Search symptoms... 377 shown

Available symptoms (selected items are highlighted in blue):

- anxiety and nervousness
- depression
- shortness of breath
- depressive or psychotic symptoms
- sharp chest pain
- dizziness
- insomnia
- abnormal involuntary movements
- chest tightness
- palpitations
- irregular heartbeat
- breathing fast
- hoarse voice
- sore throat
- difficulty speaking

Prediction Results:

Most Likely Condition

anxiety

Confidence: 87% **High Risk**

All Possibilities

Condition	Confidence
anxiety	87%
cardiomyopathy	1%
aortic valve disease	1%
atrial flutter	1%
polymyalgia rheumatica	1%

Recommendations

Consult a healthcare professional for proper diagnosis

- Suggested specialist: Psychiatrist

Figure 4.2: Health Prediction Using Logistic Regression Algorithm

Patients enter their symptoms into the system, which processes the information and predicts possible diseases using logistic regression. It then shows the most likely conditions along with their probabilities to help guide the patient.

Chapter 5: Conclusion and Future Recommendations

5.1 Conclusion

With the completion of the MediVision web application, patients can now receive disease predictions by simply inputting their symptoms, helping them make informed decisions regarding their health. Registered patients can book appointments with doctors based on urgency levels prioritized by a rating-based algorithm, ensuring critical cases are addressed promptly. Patients can also manage their profiles, view doctor details, and complete payments securely through the system. Doctors, upon logging in, can manage their schedules, view appointment requests, respond to patient needs, and track their earnings. Admins oversee and manage the activities of both doctors and patients, ensuring smooth operations across the platform. The system is developed using the MERN stack and features a user-friendly, interactive interface. MediVision meets all the project objectives and delivers a reliable, efficient, and accessible healthcare support tool for both patients and medical professionals.

5.2 Outcome

During the development of the MediVision project, many valuable lessons were learned. Working with the MERN stack helped strengthen skills in creating clean and responsive interfaces using HTML, CSS, and React, as well as building efficient APIs with Node.js, Express, and MongoDB. This process also highlighted the importance of proper planning, time management, and breaking tasks into manageable steps.

MediVision now includes several meaningful features that make the system truly helpful for both patients and doctors. The disease prediction module, powered by Logistic Regression, allows patients to quickly understand the likelihood of certain health conditions based on their symptoms. Similarly, the appointment system uses the Jaccard Similarity algorithm to prioritize critical cases, ensuring that patients who need urgent care are connected with doctors faster.

Patients can easily register, check symptoms, view predictions, and book appointments. Doctors can manage their schedules, track their earnings, and interact with patient bookings. Administrators can oversee the entire system and maintain smooth operations. Overall, MediVision offers a smart, convenient, and supportive environment for people seeking medical guidance. By combining prediction tools, intelligent prioritization, and a

smooth appointment system, it helps improve decision-making and contributes to better healthcare service delivery.

5.3 Future Recommendations

Even though MediVision already provides useful features, there are several opportunities to make the system even better in the future:

- Allow patients to chat or have video calls with doctors directly through the platform, making it easier to get quick medical advice.
- Develop a mobile app or a fully optimized mobile version so users can easily access MediVision from their phones.
- Create dashboards for doctors and admins to view helpful statistics like patient trends, system usage, and earnings over time.

References

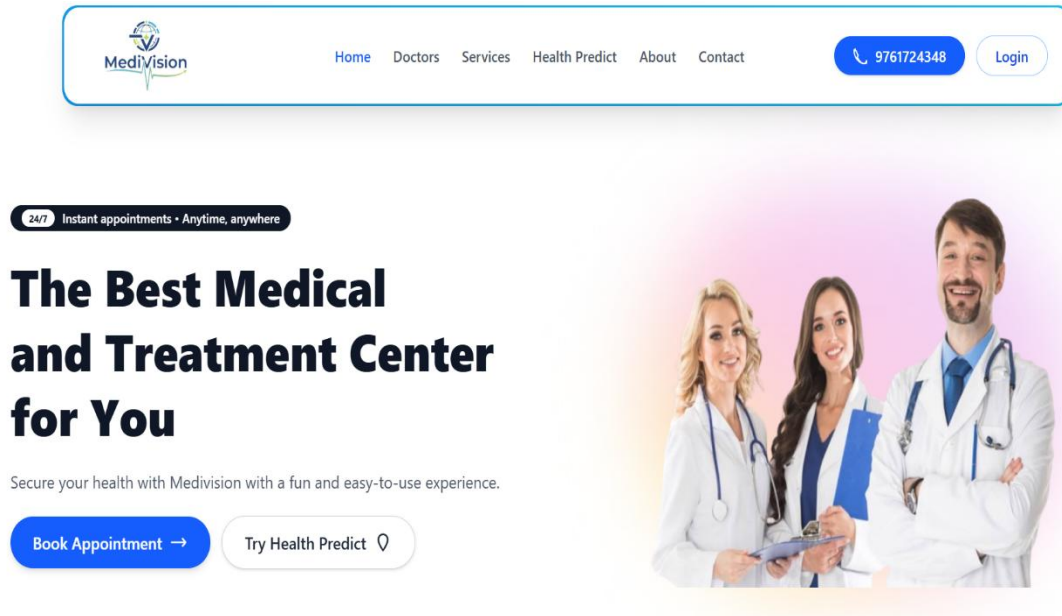
- [1] V. N. C. S. T. A. D. R. E. N. K. a. V. S. A. C. Babu, "Web Based Hospital Management System," *9th International Conference on Advanced Computing and Communication Systems (ICACCS)*, 2023 .
- [2] S. G. Jibrin Muhammad, "Web-based Clinic Management System (CMS)," *International Journal of Science and Engineering Applications*, vol. 8, no. 05, 131-135, 2019.
- [3] S. G. a. C. Sagarnal, "Disease Prediction using Machine Learning Algorithms," *International Conference for Emerging Technology (INCET)*, Belgaum, India, 2020.
- [4] S. U. a. A. M. a. Z. G. Jan, "Secure Online Doctor Appointment (Soda) Booking System," p. 63, December 29, 2024.
- [5] K. K. S. L. Priya Mangesh Nerkar, "Predictive Data Analytics Framework Based on Heart Healthcare System," *Journal of Advanced Zoology*, vol. 44, p. 3673:3686, 2023.
- [6] A. K. S. M. A. R. a. A. S. U. A. I. Newaz, "HealthGuard: A Machine Learning-Based Security Framework for Smart Healthcare Systems," *2019 Sixth International Conference on Social Networks Analysis, Management and Security (SNAMS)*, Granada, Spain, pp. 389-396, 2019.
- [7] 02 Aug 2025. [Online]. Available: <https://www.geeksforgeeks.org/machine-learning/understanding-logistic-regression>.
- [8] W. H. Organization, "Global Strategy on Digital Health 2020–2025, Geneva," Switzerland: WHO,, 2021. [Online]. Available: <https://www.who.int/publications/i/item/9789240020924>.
- [9] W. H. Organization, "Global Strategy on Digital Health 2020–2025, Geneva," 2021. [Online]. Available: <https://www.who.int/publications/i/item/9789240020924>.
- [10] P. Sharma, "Smart healthcare: the role of AI, robotics, and NLP in advancing telemedicine and remote patient monitoring," 2025. [Online]. Available: <https://bmcartificialintel.biomedcentral.com/articles/10.1186/s44398-025-00015-4>.

- [11] S. M. e. al., "Barriers and facilitators to patient utilization of noncommunicable disease services in primary healthcare facilities in Nepal: a qualitative study," *BMC Health Services Research*, 2025.
- [12] BMZ, "Digitalising Nepal's health sector – Healthy DEvelopments," *BMZ*, 2024.
- [13] 2024. [Online]. Available: <https://nepalsoft.com.np/news/view/32>.
- [14] N. Radwan, "Perceived satisfaction of web-based medical appointment systems: a systematic review," *Applied Clinical Informatics*, 2024.
- [15] J. S. Sayed K Ali, "An electronic health record system implementation in a tertiary care center: lessons learned," *PubMed Central*, 2023 Sep 20.
- [16] M. A. R. Naemi, "Design and evaluation of a web-based electronic health record for clinical use," *Frontiers in Medicine*, 04 April 2024.
- [17] M. A. Siddiqui, "Hospital management system using web technology: benefits and challenges," *BSMMU Journal*, p. 17(1), 2024.

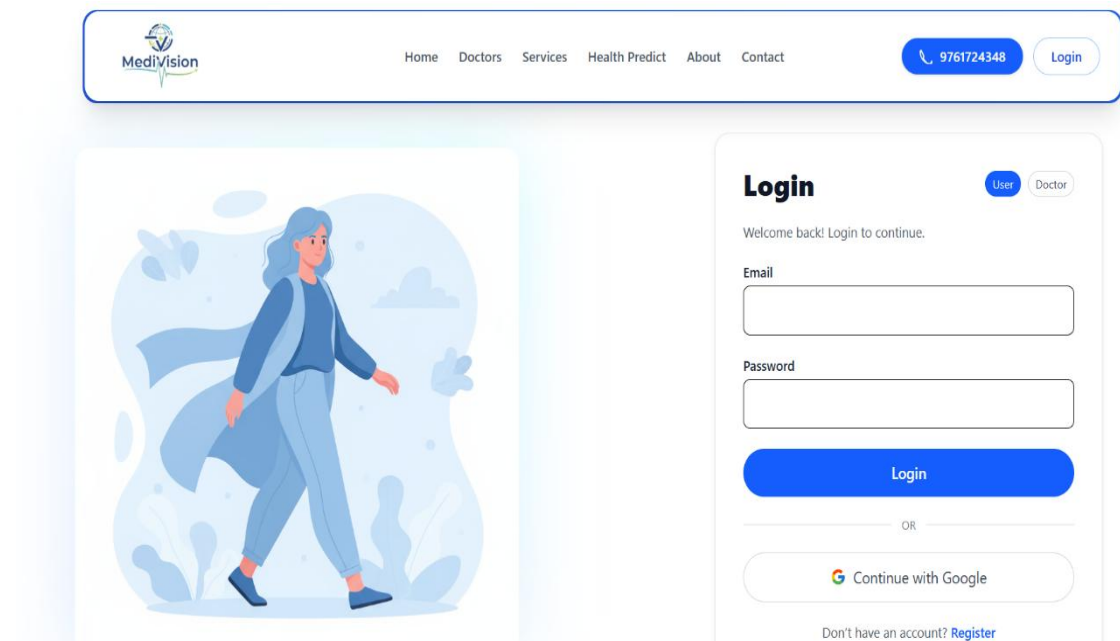
Appendices

Frontend Overview

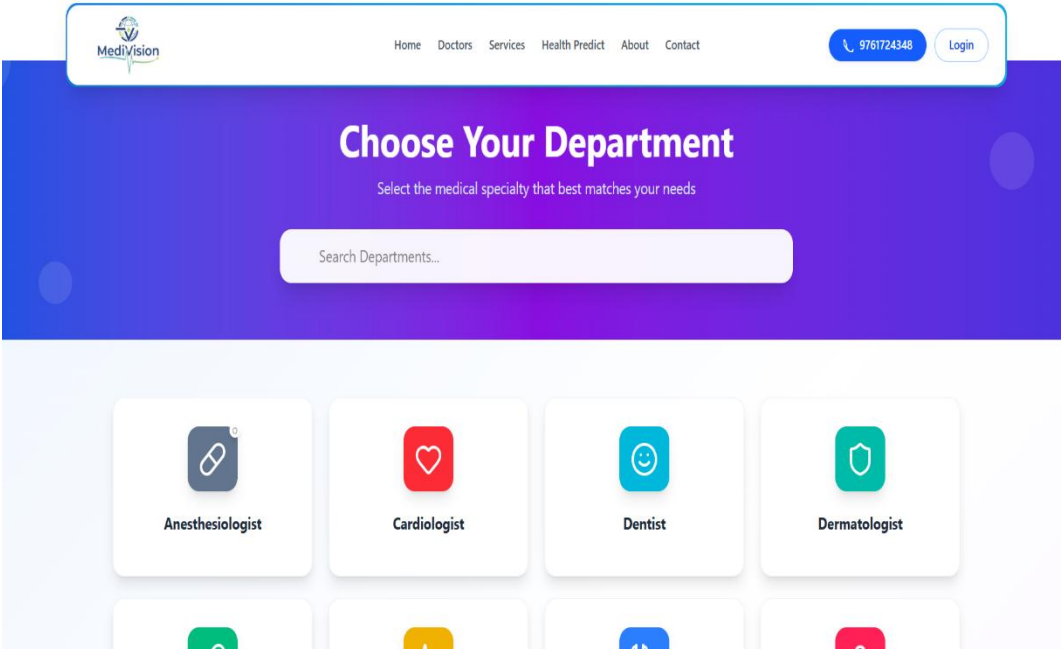
Home Page



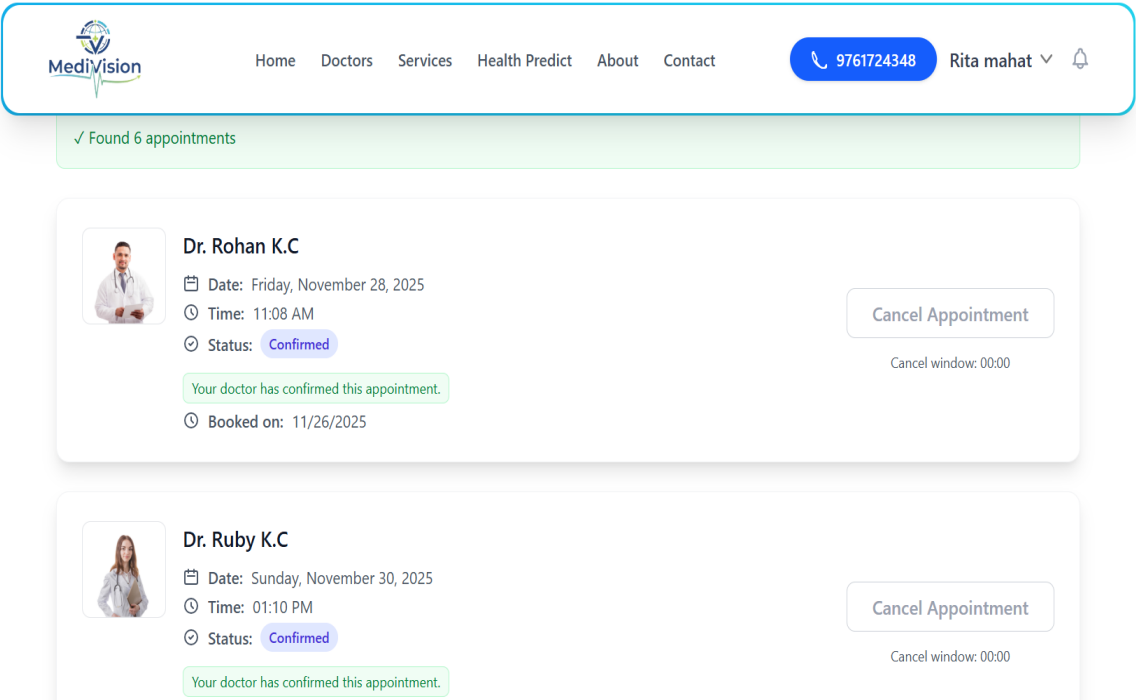
Login Page



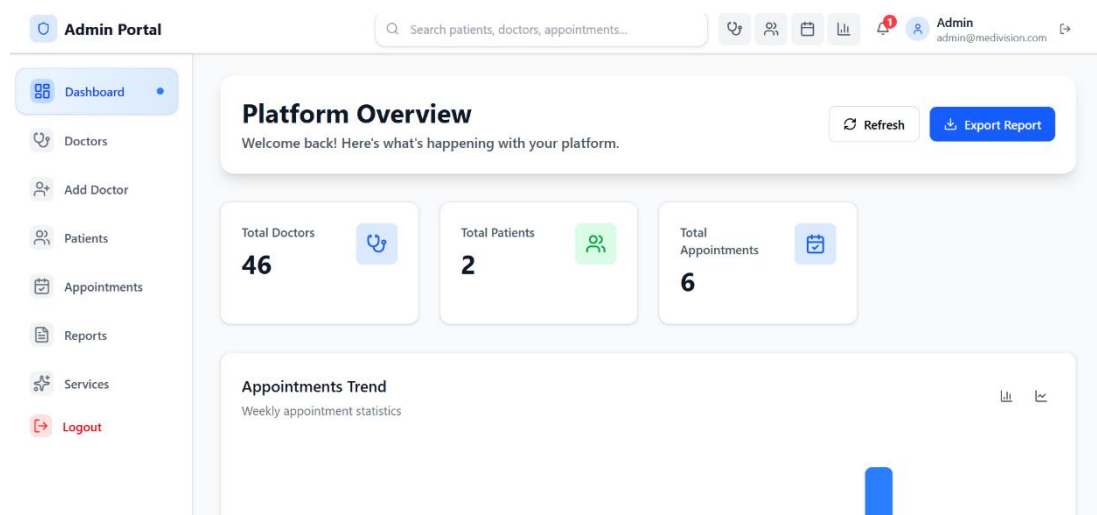
Department Page



My Appointments



Admin Dashboard



Services Page

Admin Portal Search patients, doctors, appointments... Admin admin@medivision.com

Dashboard

- Doctors
- Add Doctor
- Patients
- Appointments
- Reports
- Services**
- Logout

Services Management

Manage platform services

+ Add Service

All Services

View and manage all platform services.

Icon	Title	Description	Color	Status	Actions
Brain	Advanced Risk Prediction	Leverage advanced models for...	blue	Active	Edit Delete
Clock	Real-time Appointment Tracking	Track upcoming visits and...	orange	Active	Edit Delete
FileText	Digital Health Records	Store, access, and share...	indigo	Active	Edit Delete

Doctor Page

Admin Portal Search patients, doctors, appointments... Admin admin@medivision.com

Dashboard

- Doctors**
- Add Doctor
- Patients
- Appointments
- Reports
- Services
- Logout

Manage Doctors

Review and manage registered doctors.

Profile	Name	Speciality	Degree	Experience	Fees	Verified	Actions
	Dr. Krishab Mahat	Anesthesiologist	MBBS, MD Anesthesiologist	5 yrs	Rs. 3000	☒	Edit Deactivate Delete
	Dr. Rohan K.C	Anesthesiologist	MBBS, MD Anesthesiologist	5 yrs	Rs. 3000	☒	Edit Deactivate Delete
	Dr. Ruby K.C	Cardiologist	MBBS, MD Cardiologist	5 yrs	Rs. 5000	☒	Edit Deactivate Delete
	Dr. Aarav Sharma	Cardiologist	MBBS, MD Cardiologist	3 yrs	Rs. 5000	☒	Edit Deactivate Delete
	Dr. Suman Karki	Dentist	MBBS	2 yrs	Rs. 1000	☒	Edit Deactivate Delete

Source Code

```
import React, { useEffect, useMemo } from 'react'
import { useDispatch, useSelector } from 'react-redux'
import { useNavigate } from 'react-router-dom'
import { fetchDoctors } from '../store/doctorsSlice'
import { inferSpecialtiesFromPredictions } from '../constants/diseaseToSpecialty'
import { sortByBayesianScore } from '../utils/bayesianRanking'
const RecommendedDoctors = ({ prediction, title = 'Recommended Doctors', selectedSymptoms
= [] }) => {
  const dispatch = useDispatch()
  const navigate = useNavigate()
  const doctors = useSelector((s) => s.doctors.list)
  const isLoading = useSelector((s) => s.doctors.isLoading)
  useEffect(() => {
    if (!doctors.length && !isLoading) {
      dispatch(fetchDoctors())
    }
  }, [dispatch, doctors.length, isLoading])
  const recommendations = useMemo(() => {
    if (!prediction) return []
    const inferred = inferSpecialtiesFromPredictions(prediction).map(s => s.toLowerCase())
    if (!inferred.length) return []
    const filtered = doctors.filter(d => inferred.includes(String(d.speciality ||
)").trim().toLowerCase()))
    return sortByBayesianScore(filtered)
  }, [prediction, doctors])
  if (!prediction) return null
  return (
    <section className="bg-white rounded-2xl shadow-lg p-6 sm:p-8">
      <div className="flex items-center justify-between mb-6">
        <h3 className="text-xl font-bold text-gray-900">{title}</h3>
        <button
          onClick={() => navigate('/doctors')}
          className="text-sm font-medium text-blue-600 hover:text-blue-700"
        >
          View all →
        </button>
      </div>
    </section>
  )
}
```

```

</button>
</div>
{recommendations.length === 0 ? (
  <div className="text-sm text-gray-600 mb-4">No doctors available for this specialty right
now.</div>
) : null}
<div className="grid grid-cols-1 md:grid-cols-2 xl:grid-cols-3 gap-5">
  {recommendations.map((d) => (
    <article
      key={d._id || d.id}
      className="group border border-gray-200 rounded-xl p-4 sm:p-5 hover:shadow-md
transition-shadow flex items-center gap-4"
    >
      <div className="relative">
        <img
          src={d.image || '/placeholder-doctor.png'}
          alt={d.name}
          className="w-16 h-16 sm:w-18 sm:h-18 rounded-full object-cover ring-2 ring-blue-
50"
        />
        {d.available ? (
          <span className="absolute -bottom-1 -right-1 bg-green-500 text-white text-[10px] px-
1.5 py-0.5 rounded-full shadow">Now</span>
        ) : null}
      </div>
      <div className="flex-1 min-w-0">
        <div className="flex items-center justify-between gap-3">
          <h4 className="font-semibold text-gray-900 truncate">{d.name}</h4>
          {typeof d.fees !== 'undefined' && d.fees !== null && d.fees !== "" ? (
            <span className="text-xs bg-blue-50 text-blue-700 px-2 py-0.5 rounded">
              Rs. {d.fees}
            </span>
          ) : null}
        </div>
        <div className="text-sm text-gray-600 mt-0.5 truncate">{d.speciality}</div>
        <div className="text-xs text-gray-500 mt-1">

```

```

      {d.experience ? `${d.experience} yrs experience` : 'Experience N/A'}
      {d.isVerified ? ' • Verified' : ''}
    </div>
  </div>
  <div className="shrink-0">
    <button
      onClick={() => navigate(`/appointment/${d._id || d.id}`, {
        state: {
          fromHealthPrediction: true,
          selectedSymptoms: selectedSymptoms,
          prediction: prediction
        }
      })}
      className="px-3 py-2 text-sm bg-blue-600 text-white rounded-lg hover:bg-blue-700"
    >
      Book
    </button>
  </div>
</article>
  )))
</div>
</section>
)
}
export default RecommendedDoctors

```